AP - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continous color palettes
```

```
# load needed functions
# source(paste0(RootDir,"/src/function_library/sp_functions.R"))
# do this in the calling R script
```

```
# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste(CurPat," - ",CurTask," - ",RunLabel," - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE
```

```
CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres
```

```
# done in calling script
# print(paste0("Using root dir: ",RootDir))
# RunDir <- paste0(paste0(RootDir,"/output/",RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir,showWarnings = TRUE)
#   print(paste0("created run directory: ",RunDir))
# }
```

```
# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir,showWarnings = TRUE)
#   print(paste0("created participant directory: ",PatientDir))
# }
# TablesDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat,"/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir,showWarnings = TRUE)
#   print(paste0("created tables directory: ",TablesDir))
# }
# FigDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat,"/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir,showWarnings = TRUE)
#   print(paste0("created figures directory: ",FigDir))
# }
# ReportsDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat,"/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir,showWarnings = TRUE)
#   print(paste0("created reports directory: ",ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())
```

```
ModelDatFilename<-paste0(RootDir,"/data/",CurPat,"/",CurPat,"_",CurTask,"_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ",ModelDatFilename))
  print(sprintf("\troot dir: ",RootDir))
}
PosDat<-read.csv(ModelDatFilename)
```

```
# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values",OrigDataLen))
```

```
## [1] "Original data has 4429 values"
```

```
if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values",DataLen))
  print(sprintf("%d values eliminated.",OrigDataLen - DataLen))
}
```

```
## [1] "Data without fractional preserved values has 4389 values"
## [1] "40 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite

# get counts of syllable components in different serial positions
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syl

syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))

kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	554	35	130	NA	NA	719
2	67	NA	440	98	108	713
3	315	NA	169	212	16	712
4	306	NA	241	69	39	655
5	234	NA	213	73	39	559
6	211	1	142	70	22	446
7	179	NA	103	29	18	329
8	93	NA	53	24	3	173
9	74	NA	2	NA	7	83

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7705146	0.0486787	0.1808067	NA	NA	719
2	0.0939691	NA	0.6171108	0.1374474	0.1514727	713
3	0.4424157	NA	0.2373596	0.2977528	0.0224719	712
4	0.4671756	NA	0.3679389	0.1053435	0.0595420	655
5	0.4186047	NA	0.3810376	0.1305903	0.0697674	559
6	0.4730942	0.0022422	0.3183857	0.1569507	0.0493274	446
7	0.5440729	NA	0.3130699	0.0881459	0.0547112	329
8	0.5375723	NA	0.3063584	0.1387283	0.0173410	173
9	0.8915663	NA	0.0240964	NA	0.0843373	83

```
# get percentages only from summary table
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
syll_comp_perc <- syll_comp_perc %>% rename("Coda" ="1","Satellite"="S")

# pivot to long format for plotting
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
            names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda","Sa
```

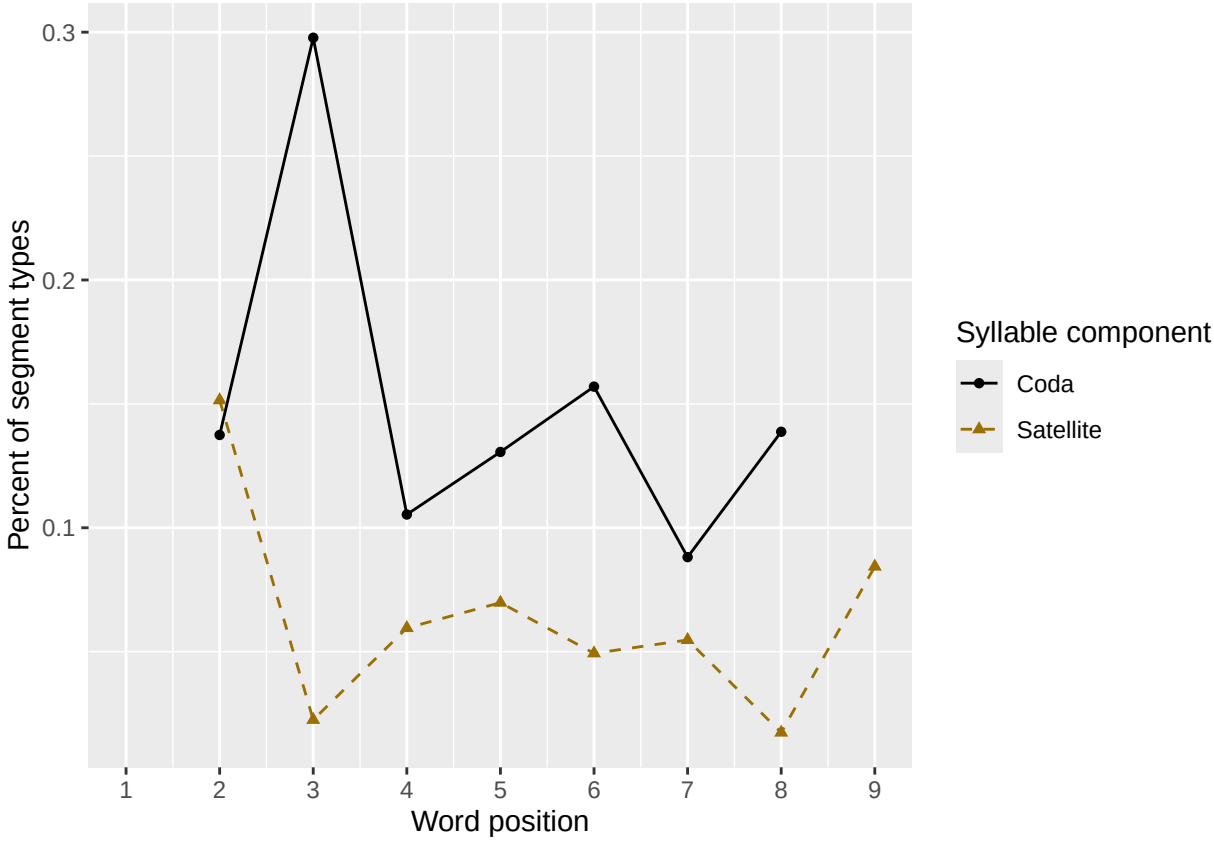
```
# plot satellite and coda occurances across position

syll_comp_plot <- ggplot(syll_comp_plot_data,
                         aes(x=pos,y=percent,group=syll_component,
                             color=syll_component,
                             linetype = syll_component,
                             shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype_
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_pos_of_syll_complexities_in_stimuli.png"),plot=sy
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`)
```

```
syll_comp_plot
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
## Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```




```
# len/pos table
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```
min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,"_preserved_percentages_by_len_
pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`    `4`    `5`    `6`    `7`    `8`    `9`
##     <int> <dbl> <dbl> <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
## 1       4 0.917 0.983 0.983 NA     NA     NA     NA     NA     NA
## 2       5 0.926 0.947 0.968  0.958 NA     NA     NA     NA     NA
## 3       6 0.916 0.966 0.958  0.950  0.966 NA     NA     NA     NA
## 4       7 0.913 0.974 0.956  0.965  0.965  0.991 NA     NA     NA
## 5       8 0.889 0.967 0.973  0.954  0.954  0.955  0.948 NA     NA
## 6       9 0.957 0.967 0.925  0.935  0.925  0.947  0.957  0.946 NA
## 7      10 0.928 0.976 0.952  0.940  0.976  0.928  0.976  0.963  0.928
```

```
# len/pos table
pos_len_N <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved),N=n()) %>% dply
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                 "_N_by_len_position.csv"))
pos_len_N_table
```

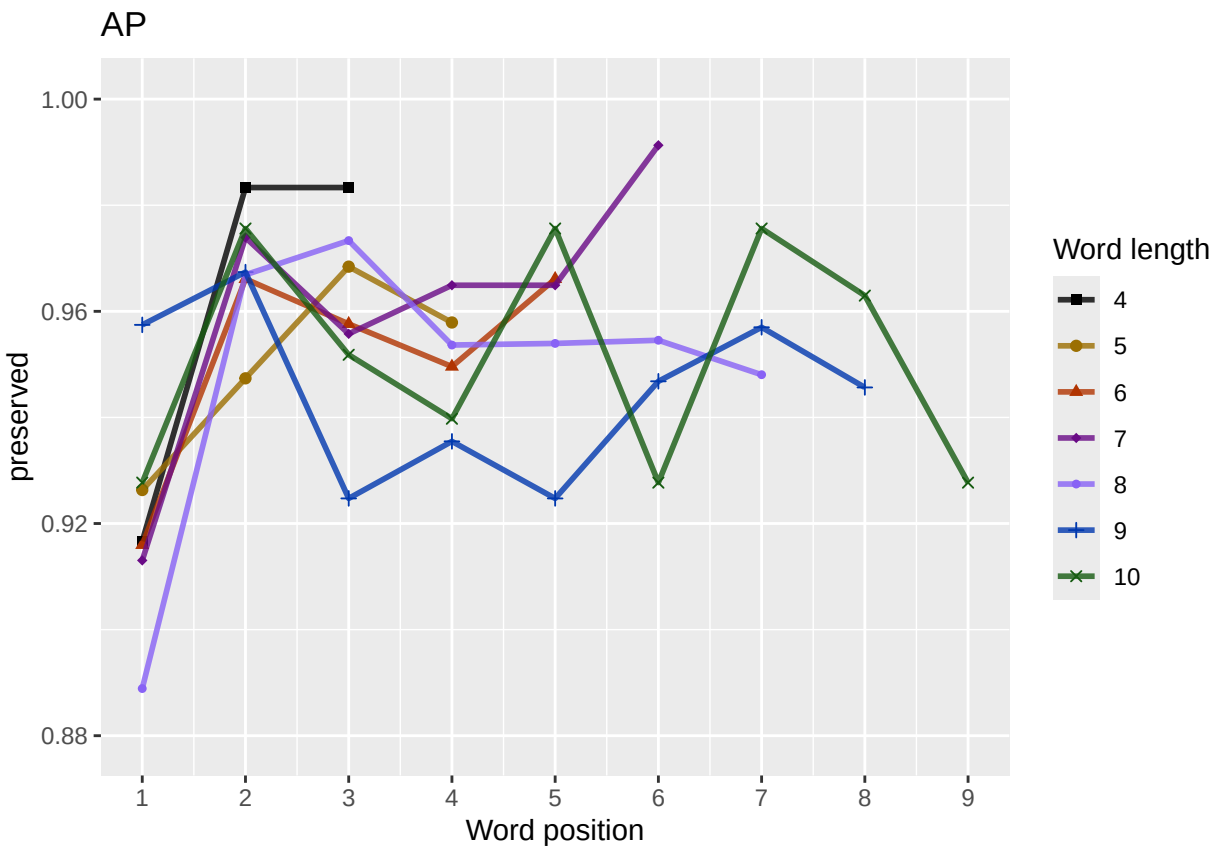
```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##     <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1       4    60    60    60    NA    NA    NA    NA    NA    NA
## 2       5    95    95    95    95    NA    NA    NA    NA    NA
## 3       6   119   118   118   119   118    NA    NA    NA    NA
## 4       7   115   115   113   114   114   115    NA    NA    NA
## 5       8   153   151   150   151   152   154   154    NA    NA
## 6       9    94    92    93    93    93    94    93    92    NA
## 7      10    83    82    83    83    82    83    82    81    83
```

```
obs_linetypes <- c("solid","solid","solid","solid",
                   "solid","solid","solid","solid","solid")
pred_linetypes <- "dashed"
pos_len_summary$stimlen<-factor(pos_len_summary$stimlen)
pos_len_summary$pos<-factor(pos_len_summary$pos)
# len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color=stimle
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
                                           paste0(CurPat),
                                           NULL,
                                           c(min_preserved,max_preserved),
                                           palette_values,
                                           shape_values,
                                           obs_linetypes,
                                           pred_linetypes)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot,device="png",unit="cm",width=15,height=11)
len_pos_plot
```



Length and position

```
# length and position

LPModelEquations<-c("preserved ~ 1",
            "preserved ~ stimlen",
            "preserved ~ pos",
            "preserved ~ stimlen + pos",
            "preserved ~ stimlen*pos",
            "preserved ~ I(pos^2)+pos",
            "preserved ~ stimlen + I(pos^2) + pos",
            "preserved ~ stimlen * (I(pos^2) + pos)"
)

LPRes<-TestModels(LPModelEquations,PosDat)
```

```
## ****************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
```

```
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     2.24753     -0.04042      0.39793
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4386 Residual
## Null Deviance:       1710
## Residual Deviance: 1699  AIC: 1705
## log likelihood:  -849.7454
## Nagelkerke R2:  0.007642074
## % pres/err predicted correctly:  -405.0016
## % of predictable range [ (model-null)/(1-null) ]:  0.002775195
## ***************************
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos
##     2.55549     -0.04232     -0.03818      0.39139
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4385 Residual
## Null Deviance:       1710
## Residual Deviance: 1699  AIC: 1707
## log likelihood:  -849.3087
## Nagelkerke R2:  0.008257024
## % pres/err predicted correctly:  -404.9485
## % of predictable range [ (model-null)/(1-null) ]:  0.002905711
## ***************************
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          I(pos^2)               pos  stimlen:I(pos^2)       stiml
##          0.85572           0.17206          -0.17511           1.53540           0.01631          -0
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4383 Residual
## Null Deviance:       1710
## Residual Deviance: 1695  AIC: 1707
## log likelihood:  -847.6753
## Nagelkerke R2:  0.01055637
## % pres/err predicted correctly:  -404.6227
## % of predictable range [ (model-null)/(1-null) ]:  0.003705955
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
```

```
##     1.91036      0.09192      0.52149     -0.05258
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4385 Residual
## Null Deviance:       1710
## Residual Deviance: 1700  AIC: 1708
## log likelihood:  -850.1147
## Nagelkerke R2:  0.007121847
## % pres/err predicted correctly:  -405.1794
## % of predictable range [ (model-null)/(1-null) ]:  0.002338465
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      2.7682       0.0545
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4387 Residual
## Null Deviance:       1710
## Residual Deviance: 1708  AIC: 1712
## log likelihood:  -853.8292
## Nagelkerke R2:  0.0018847
## % pres/err predicted correctly:  -405.8631
## % of predictable range [ (model-null)/(1-null) ]:  0.0006591293
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##     3.17673     -0.06168      0.07221
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4386 Residual
## Null Deviance:       1710
## Residual Deviance: 1706  AIC: 1712
## log likelihood:  -852.8644
## Nagelkerke R2:  0.003245846
## % pres/err predicted correctly:  -405.6968
## % of predictable range [ (model-null)/(1-null) ]:  0.001067609
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       2.971
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4388 Residual
```

```
## Null Deviance:       1710
## Residual Deviance: 1710  AIC: 1712
## log likelihood:  -855.1644
## Nagelkerke R2:  -6.880197e-16
## % pres/err predicted correctly:  -406.1315
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##     3.18715     -0.02811
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4387 Residual
## Null Deviance:       1710
## Residual Deviance: 1710  AIC: 1714
## log likelihood:  -854.9369
## Nagelkerke R2:  0.0003212711
## % pres/err predicted correctly:  -406.0896
## % of predictable range [ (model-null)/(1-null) ]:  0.0001027109
## ***************************
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                         AIC=LPRes$AIC,
                         row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                              "_len_pos_model_summary.csv"),row.names = FALSE)
kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:pos	I(pos^2)	stimlen:I(pos^2)
preserved ~ I(pos^2) + pos	1705.490	0.000000	1.0000000	0.4244464	0.0076421	2.475288	NA	0.3979344	NA	-0.0404221	NA
preserved ~ stimlen + I(pos^2) + pos	1706.617	1.126707	0.5692969	0.2416360	0.0082572	2.555488	-0.0423187	0.3913865	NA	-0.0381829	NA
preserved ~ stimlen * (I(pos^2) + pos)	1707.351	1.859856	0.3945822	0.1674790	0.0105564	2.855716	-0.1720627	2.5354032	-0.1399350	-0.1751118	0.0163056
preserved ~ stimlen * pos	1708.222	2.731864	0.2542796	0.1079281	0.0071218	2.910361	-0.0919193	3.5214927	-0.0525769	NA	NA

Model	AIC	DeltaAIC	AICexpAICwt	NagR2	(Intercept)	stimlen	pos	stimlen:pos	I(pos^2)	stimlen:I(pos^2)
preserved ~ pos	1711.656	0.167660	1.045783	0.6019432	0.7001884	2.7682376	NA	0.0545009	NA	NA	NA
preserved ~ stimlen + pos	1711.729	0.238050	1.044200	0.2018760	0.6003245	3.1767257	- 0.0616753	0.0722129	NA	NA	NA
preserved ~ 1	1712.329	0.838079	0.032748	9.0013898	0.0000000	2.0708936	NA	NA	NA	NA	NA
preserved ~ stimlen	1713.878	2.382987	0.015123	4.1706419	0.0003233	2.1871468	- 0.0281072	NA	NA	NA	NA

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ I(pos^2) + pos"
```

```
print(BestLPModel)
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     2.24753     -0.04042      0.39793
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4386 Residual
## Null Deviance:       1710
## Residual Deviance: 1699  AIC: 1705
```

```
PosDat$LPFitted<-fitted(BestLPModel)
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],
                        NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)
fitted_pos_len_table
```

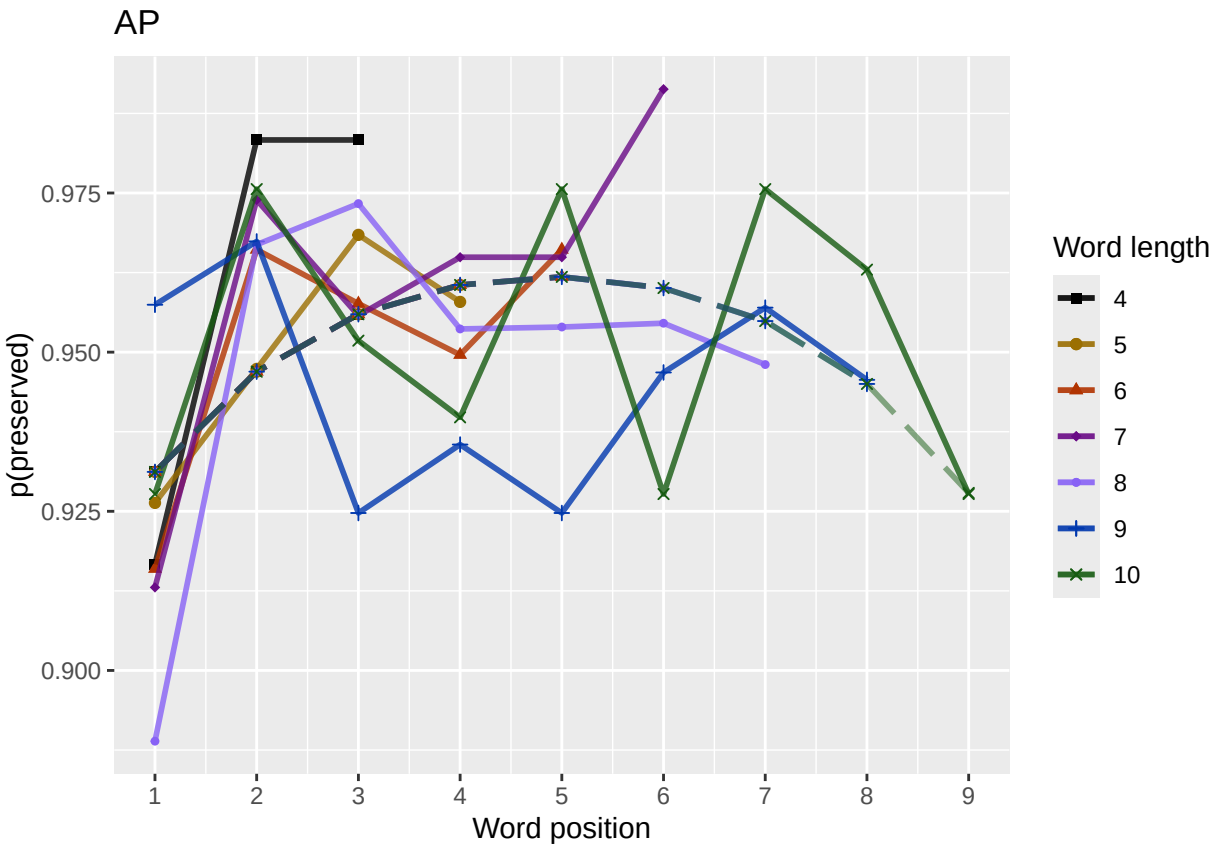
```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`    `4`    `5`    `6`    `7`    `8`    `9`
##     <int> <dbl> <dbl> <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
## 1       4 0.931 0.947 0.956 NA     NA     NA     NA     NA     NA
## 2       5 0.931 0.947 0.956  0.961 NA     NA     NA     NA     NA
## 3       6 0.931 0.947 0.956  0.961  0.962 NA     NA     NA     NA
## 4       7 0.931 0.947 0.956  0.961  0.962  0.960 NA     NA     NA
## 5       8 0.931 0.947 0.956  0.961  0.962  0.960  0.955 NA     NA
## 6       9 0.931 0.947 0.956  0.961  0.962  0.960  0.955  0.945 NA
## 7      10 0.931 0.947 0.956  0.961  0.962  0.960  0.955  0.945  0.928
```

```
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color
```

```
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitte
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(pas

fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
                                          paste0(PosDat$patient[1]),
                                          "LPFitted",
                                          NULL,
                                          palette_values,
                                          shape_values,
                                          obs_linetypes,
                                          pred_linetypes = c("longdash")
                                          )
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_percent_preserved_by_length_pos_wfit.png"),plot=
fitted_len_pos_plot
```



length and position without fragments to see if this changes position^2 influence

```
# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models

# number responses
resp_num<-0
prev_pos<-9999 # big number to initialize (so first position is smaller)
```

```
resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array
```

```
# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag
```

```
## # A tibble: 1 x 2
##   frag_sum     N
##      <int> <int>
## 1        8   721
```

```
num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100
```

```
## Warning: Unknown or uninitialised column: `percent_with_frag`.
```

```
print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
              num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))
```

```
## [1] "The number of responses with fragments was 8 / 721 = 1.11 percent"
```

```
write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                          CurTask,"_percent_fragments.csv"),row.names = FALSE)
```

```
# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)
```

```
## ****************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##     1.59913      0.11567      0.72070     -0.06987
##
## Degrees of Freedom: 4373 Total (i.e. Null);  4370 Residual
## Null Deviance:       1619
## Residual Deviance: 1599  AIC: 1607
## log likelihood:  -799.352
## Nagelkerke R2:  0.01472895
## % pres/err predicted correctly:  -377.1601
## % of predictable range [ (model-null)/(1-null) ]:  0.004560486
## ****************************
## model index:  6
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     2.17155     -0.04149      0.44569
##
## Degrees of Freedom: 4373 Total (i.e. Null);  4371 Residual
## Null Deviance:       1619
## Residual Deviance: 1603  AIC: 1609
## log likelihood:  -801.4103
## Nagelkerke R2:  0.01169869
## % pres/err predicted correctly:  -377.3352
## % of predictable range [ (model-null)/(1-null) ]:  0.004099522
## ***************************
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos
##     2.61617     -0.06083     -0.03827      0.43610
##
## Degrees of Freedom: 4373 Total (i.e. Null);  4370 Residual
## Null Deviance:       1619
## Residual Deviance: 1601  AIC: 1609
## log likelihood:  -800.5436
## Nagelkerke R2:  0.01297504
## % pres/err predicted correctly:  -377.2387
## % of predictable range [ (model-null)/(1-null) ]:  0.004353597
## ***************************
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          I(pos^2)               pos  stimlen:I(pos^2)       stiml
##          1.00782           0.15873          -0.10678           1.31603           0.00986          -0
##
## Degrees of Freedom: 4373 Total (i.e. Null);  4368 Residual
## Null Deviance:       1619
## Residual Deviance: 1597  AIC: 1609
## log likelihood:  -798.5489
## Nagelkerke R2:  0.01591056
## % pres/err predicted correctly:  -376.9068
## % of predictable range [ (model-null)/(1-null) ]:  0.005227259
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
```

```
##
## Coefficients:
## (Intercept)      stimlen          pos
##     3.20263     -0.07848      0.12268
##
## Degrees of Freedom: 4373 Total (i.e. Null);  4371 Residual
## Null Deviance:       1619
## Residual Deviance: 1607  AIC: 1613
## log likelihood:  -803.6963
## Nagelkerke R2:  0.008329811
## % pres/err predicted correctly:  -377.8914
## % of predictable range [ (model-null)/(1-null) ]:  0.002635415
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      2.6773       0.1012
##
## Degrees of Freedom: 4373 Total (i.e. Null);  4372 Residual
## Null Deviance:       1619
## Residual Deviance: 1610  AIC: 1614
## log likelihood:  -805.1941
## Nagelkerke R2:  0.006120623
## % pres/err predicted correctly:  -378.1201
## % of predictable range [ (model-null)/(1-null) ]:  0.00203342
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       3.044
##
## Degrees of Freedom: 4373 Total (i.e. Null);  4373 Residual
## Null Deviance:       1619
## Residual Deviance: 1619  AIC: 1621
## log likelihood:  -809.3384
## Nagelkerke R2:  -7.178649e-16
## % pres/err predicted correctly:  -378.8926
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
```

```
##     3.23210     -0.02452
##
## Degrees of Freedom: 4373 Total (i.e. Null);  4372 Residual
## Null Deviance:       1619
## Residual Deviance: 1618  AIC: 1622
## log likelihood:  -809.1764
## Nagelkerke R2:  0.0002394687
## % pres/err predicted correctly:  -378.8644
## % of predictable range [ (model-null)/(1-null) ]:  7.41428e-05
## ***************************
```

```
NoFragBestLPModel<-NoFrag_LPRes$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPRes$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPRes$Model,
                        AIC=NoFrag_LPRes$AIC,
                        row.names = NoFrag_LPRes$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPRes$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPRes$CoefficientValues, by='row.names',sort=FALSE
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:pos	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * pos	1606.704	0.000000	1.000000	0.4972672	0.0147283	1.5599131	0.1156702	7.7206994	-0.0698735	NA	NA
preserved ~ I(pos^2) + pos	1608.821	2.116561	0.3470521	0.1725776	0.0116987	1.7171554	NA	0.4456867	NA	-0.0414873	NA
preserved ~ stimlen + I(pos^2) + pos	1609.087	2.383130	0.3037406	0.1510423	0.0129752	1.6616170	-0.0608298	0.4361043	NA	-0.0382697	NA
preserved ~ stimlen * (I(pos^2) + pos)	1609.098	2.393754	0.3021364	0.1502425	0.0159106	1.6007817	0.1587256	3.1602900	-0.1202530	-0.1067800	0.0098598
preserved ~ stimlen + pos	1613.396	6.688594	0.0352850	0.0175461	0.0083298	1.8202627	-0.0784800	0.1226770	NA	NA	NA
preserved ~ pos	1614.387	7.684173	0.0214488	0.0106658	0.0061206	1.6677344	NA	0.1012416	NA	NA	NA
preserved ~ 1	1620.677	13.972805	0.0009240	0.0004597	0.0000000	1.3043565	NA	NA	NA	NA	NA
preserved ~ stimlen	1622.353	15.648807	0.0003999	0.0001988	0.0002395	1.5232101	-0.0245208	NA	NA	NA	NA

```
# plot no fragment data

NoFragData$LPFitted<-fitted(NoFragBestLPModel)
```

```
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
                          NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

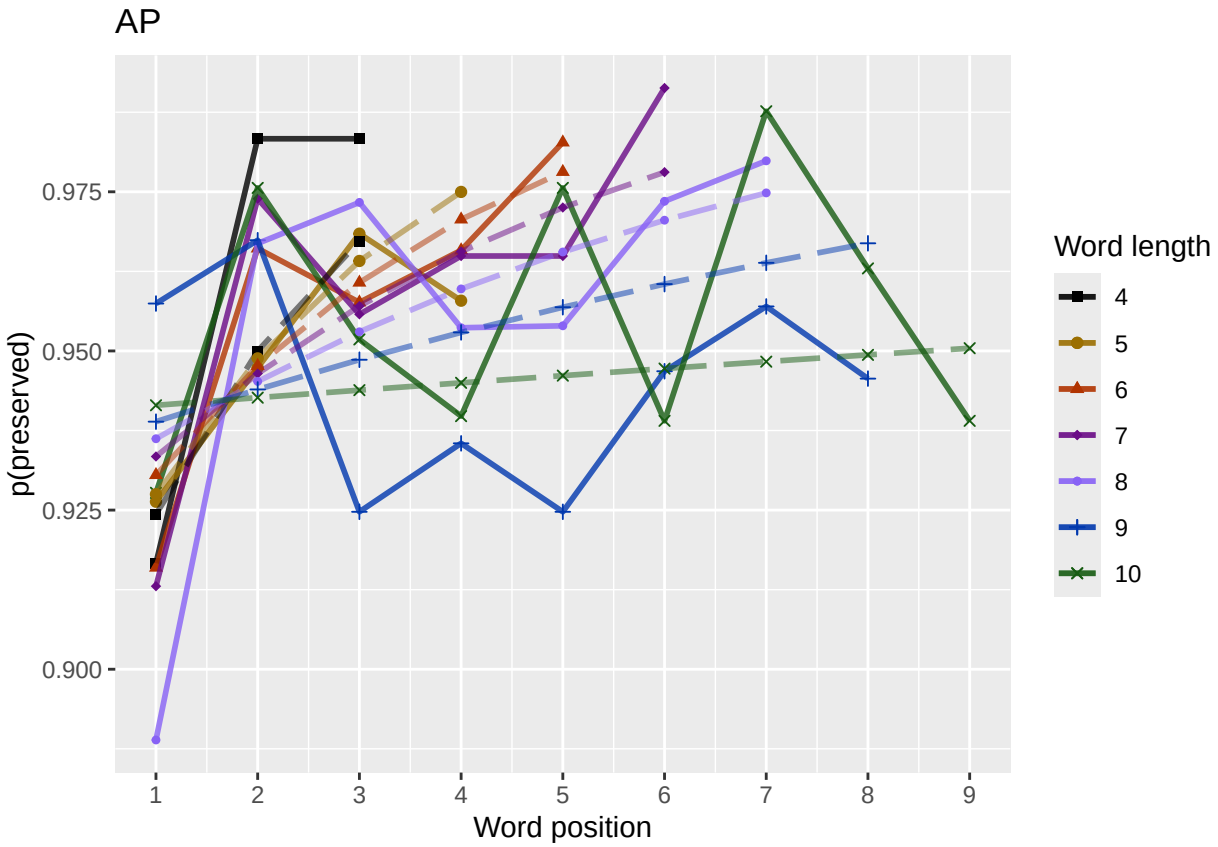
```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`    `4`    `5`    `6`    `7`    `8`    `9`
##     <int> <dbl> <dbl> <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
## 1       4 0.924 0.950 0.967 NA     NA     NA     NA     NA     NA
## 2       5 0.927 0.949 0.964  0.975 NA     NA     NA     NA     NA
## 3       6 0.931 0.948 0.961  0.971  0.978 NA     NA     NA     NA
## 4       7 0.933 0.946 0.957  0.966  0.973  0.978 NA     NA     NA
## 5       8 0.936 0.945 0.953  0.960  0.966  0.971  0.975 NA     NA
## 6       9 0.939 0.944 0.949  0.953  0.957  0.961  0.964  0.967 NA
## 7      10 0.941 0.943 0.944  0.945  0.946  0.947  0.948  0.949  0.950
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitte
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(pas

nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
                                        paste0(NoFragData$patient[1]),
                                        "LPFitted",
                                        NULL,
                                        palette_values,
                                        shape_values,
                                        obs_linetypes,
                                        pred_linetypes = c("longdash")
                                        )
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
              "no_fragments_percent_preserved_by_length_pos_wfit.png"),
       plot=nofrag_fitted_len_pos_plot,
       device="png",unit="cm",
       width=15,height=11)
nofrag_fitted_len_pos_plot
```



back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.88 - 1.00"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account

# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
table_to_use <- fitted_pos_len_table
# take away column with length information
table_to_use <- table_to_use[,2:ncol(table_to_use)]

# take averages for positions
# don't want downward estimates influenced by return upward of U
# therefore, for downward influence, use only the values before the min
# take the difference between each value (differences between position proportion correct) **NOTE** pro
# average the difference in probabilities

# in case min is close to the end or we are not using a min (for non-U-shaped)
# functions, we need to get differences _first_ and then average those (e.g.
# if there is an effect of length and we just average position data,
# later positions) will have a lower average (because of the length effect)
# even if all position functions go upward
```

```
table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)
```

```
## [1] "mean change in probability for each additional length: "
```

```
print(OA_mean_len_diff)
```

```
## [1] 0
```

```
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding  U-shape): "
print(CurrentLabel)
```

```
## [1] "mean change in probability for each additional position (excluding  U-shape): "
```

```
print(OA_mean_pos_diff)
```

```
## [1] 0.007659894
```

```
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)
```

```
if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
                               2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
```

```
    CurrentLabel<-"Average upward change after U minimum"
    print(CurrentLabel)
    print(OA_mean_pos_u_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

    CurrentLabel<-"Proportion of average downward change"
    prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
  }
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}
```

```
## [1] "No U-shape in this participant"
```

```
if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upwa
```

```
  CurrentLabel<-"return upward value"
  print(upward_dist[biggest_return_upward_row])
   results_report_DF <- AddReportLine(results_report_DF,
                                      CurrentLabel,
                                      upward_dist[biggest_return_upward_row])

  print(" ")
  # percentage return upward
  percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upwar
  CurrentLabel <- "Return upward as a proportion of the downward distance:"
  print(CurrentLabel)
  print(percentage_return_upward)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                         percentage_return_upward)
}else{
  print("no U-shape in this participant")
}
```

```
## [1] "no U-shape in this participant"
```

```
FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small
            "preserved ~ stimlen*log_freq",
            "preserved ~ stimlen+log_freq",
            "preserved ~ pos*log_freq",
            "preserved ~ pos+log_freq",
            "preserved ~ stimlen*log_freq + pos*log_freq",
            "preserved ~ stimlen*log_freq + pos",
            "preserved ~ stimlen + pos*log_freq",
            "preserved ~ stimlen + pos + log_freq",
            "preserved ~ (I(pos^2)+pos)*log_freq",
            "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
            "preserved ~ stimlen*log_freq + I(pos^2) + pos",
            "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
            "preserved ~ stimlen + I(pos^2) + pos + log_freq",

            # models without frequency
            "preserved ~ 1",
            "preserved ~ stimlen",
            "preserved ~ pos",
            "preserved ~ stimlen + pos",
            "preserved ~ stimlen*pos",
            "preserved ~ I(pos^2)+pos",
            "preserved ~ stimlen + I(pos^2) + pos",
            "preserved ~ stimlen * (I(pos^2) + pos)"
)

FLPRes<-TestModels(FLPModelEquations,PosDat)
```

```
## ***************************
## model index:  13
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
```

```
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos     log_freq
##    2.218823     0.004615    -0.038863     0.396946     0.146144
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4384 Residual
## Null Deviance:       1710
## Residual Deviance: 1684  AIC: 1694
## log likelihood:  -842.0563
## Nagelkerke R2:  0.01845304
## % pres/err predicted correctly:  -403.6239
## % of predictable range [ (model-null)/(1-null) ]:  0.006159158
## **************************
## model index:  9
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##       (Intercept)           I(pos^2)                pos           log_freq  I(pos^2):log_freq       l
##          2.197330          -0.043065           0.435407          -0.000975          -0.009123
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4383 Residual
## Null Deviance:       1710
## Residual Deviance: 1682  AIC: 1694
## log likelihood:  -841.1471
## Nagelkerke R2:  0.01972894
## % pres/err predicted correctly:  -403.4922
## % of predictable range [ (model-null)/(1-null) ]:  0.006482651
## **************************
## model index:  11
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq          I(pos^2)               pos  stimlen:log
##         2.216591          0.003563          0.210127         -0.039147          0.399222         -0.(
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4383 Residual
## Null Deviance:       1710
## Residual Deviance: 1684  AIC: 1696
## log likelihood:  -841.9853
## Nagelkerke R2:  0.01855266
## % pres/err predicted correctly:  -403.6191
## % of predictable range [ (model-null)/(1-null) ]:  0.006170969
## **************************
## model index:  12
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##       (Intercept)           stimlen           I(pos^2)               pos           log_freq  I(pos
##         2.171e+00          3.562e-03         -4.326e-02          4.360e-01          9.439e-05
```

```
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4382 Residual
## Null Deviance:       1710
## Residual Deviance: 1682  AIC: 1696
## log likelihood:  -841.1443
## Nagelkerke R2:  0.01973286
## % pres/err predicted correctly:  -403.4885
## % of predictable range [ (model-null)/(1-null) ]:  0.006491574
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos     log_freq
##     2.74906      0.06848      0.14848
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4386 Residual
## Null Deviance:       1710
## Residual Deviance: 1692  AIC: 1698
## log likelihood:  -845.7631
## Nagelkerke R2:  0.01324597
## % pres/err predicted correctly:  -404.3658
## % of predictable range [ (model-null)/(1-null) ]:  0.004336761
## ***************************
## model index:  10
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##       (Intercept)            stimlen           log_freq           I(pos^2)                pos   stim
##          2.177042           0.001138           0.077349          -0.043015           0.436267
##      log_freq:pos
##          0.086098
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4381 Residual
## Null Deviance:       1710
## Residual Deviance: 1682  AIC: 1698
## log likelihood:  -841.0444
## Nagelkerke R2:  0.01987296
## % pres/err predicted correctly:  -403.4802
## % of predictable range [ (model-null)/(1-null) ]:  0.006512159
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##  (Intercept)           pos      log_freq  pos:log_freq
##      2.72024       0.07880       0.09334       0.01554
##
```

```
## Degrees of Freedom: 4388 Total (i.e. Null);  4385 Residual
## Null Deviance:       1710
## Residual Deviance: 1691  AIC: 1699
## log likelihood:  -845.3805
## Nagelkerke R2:  0.01378388
## % pres/err predicted correctly:  -404.3343
## % of predictable range [ (model-null)/(1-null) ]:  0.004414207
## **************************
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos     log_freq
##     2.85194     -0.01538      0.07241      0.14508
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4385 Residual
## Null Deviance:       1710
## Residual Deviance: 1691  AIC: 1699
## log likelihood:  -845.7083
## Nagelkerke R2:  0.01332297
## % pres/err predicted correctly:  -404.3616
## % of predictable range [ (model-null)/(1-null) ]:  0.00434712
## **************************
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##  (Intercept)       stimlen           pos      log_freq  pos:log_freq
##      2.84617      -0.01895       0.08396       0.08709       0.01612
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4384 Residual
## Null Deviance:       1710
## Residual Deviance: 1691  AIC: 1701
## log likelihood:  -845.2979
## Nagelkerke R2:  0.01389999
## % pres/err predicted correctly:  -404.3286
## % of predictable range [ (model-null)/(1-null) ]:  0.004428204
## **************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq               pos  stimlen:log_freq
##          2.85378          -0.01623           0.18721           0.07241          -0.00545
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4384 Residual
## Null Deviance:       1710
## Residual Deviance: 1691  AIC: 1701
```

```
## log likelihood:  -845.6784
## Nagelkerke R2:  0.01336509
## % pres/err predicted correctly:  -404.3528
## % of predictable range [ (model-null)/(1-null) ]:  0.004368764
## **************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     log_freq
##     2.86256      0.01826      0.14497
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4386 Residual
## Null Deviance:       1710
## Residual Deviance: 1696  AIC: 1702
## log likelihood:  -847.7839
## Nagelkerke R2:  0.01040354
## % pres/err predicted correctly:  -404.7567
## % of predictable range [ (model-null)/(1-null) ]:  0.003376744
## **************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq               pos  stimlen:log_freq      log_fr
##          2.84781          -0.02164           0.18322           0.08639          -0.01436           0
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4383 Residual
## Null Deviance:       1710
## Residual Deviance: 1690  AIC: 1702
## log likelihood:  -845.1155
## Nagelkerke R2:  0.01415628
## % pres/err predicted correctly:  -404.3001
## % of predictable range [ (model-null)/(1-null) ]:  0.004498191
## **************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq  stimlen:log_freq
##         2.864406          0.017405          0.186886         -0.005421
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4385 Residual
## Null Deviance:       1710
## Residual Deviance: 1696  AIC: 1704
## log likelihood:  -847.7542
## Nagelkerke R2:  0.01044529
## % pres/err predicted correctly:  -404.7474
```

```
## % of predictable range [ (model-null)/(1-null) ]:  0.00339966
## ***************************
## model index:  19
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     2.24753     -0.04042      0.39793
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4386 Residual
## Null Deviance:       1710
## Residual Deviance: 1699  AIC: 1705
## log likelihood:  -849.7454
## Nagelkerke R2:  0.007642074
## % pres/err predicted correctly:  -405.0016
## % of predictable range [ (model-null)/(1-null) ]:  0.002775195
## ***************************
## model index:  20
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos
##     2.55549     -0.04232     -0.03818      0.39139
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4385 Residual
## Null Deviance:       1710
## Residual Deviance: 1699  AIC: 1707
## log likelihood:  -849.3087
## Nagelkerke R2:  0.008257024
## % pres/err predicted correctly:  -404.9485
## % of predictable range [ (model-null)/(1-null) ]:  0.002905711
## ***************************
## model index:  21
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          I(pos^2)               pos  stimlen:I(pos^2)       stiml
##          0.85572           0.17206          -0.17511           1.53540           0.01631          -0
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4383 Residual
## Null Deviance:       1710
## Residual Deviance: 1695  AIC: 1707
## log likelihood:  -847.6753
## Nagelkerke R2:  0.01055637
## % pres/err predicted correctly:  -404.6227
## % of predictable range [ (model-null)/(1-null) ]:  0.003705955
## ***************************
## model index:  18
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##     1.91036      0.09192      0.52149     -0.05258
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4385 Residual
## Null Deviance:       1710
## Residual Deviance: 1700  AIC: 1708
## log likelihood:  -850.1147
## Nagelkerke R2:  0.007121847
## % pres/err predicted correctly:  -405.1794
## % of predictable range [ (model-null)/(1-null) ]:  0.002338465
## ***************************
## model index:  16
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      2.7682       0.0545
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4387 Residual
## Null Deviance:       1710
## Residual Deviance: 1708  AIC: 1712
## log likelihood:  -853.8292
## Nagelkerke R2:  0.0018847
## % pres/err predicted correctly:  -405.8631
## % of predictable range [ (model-null)/(1-null) ]:  0.0006591293
## ***************************
## model index:  17
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##     3.17673     -0.06168      0.07221
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4386 Residual
## Null Deviance:       1710
## Residual Deviance: 1706  AIC: 1712
## log likelihood:  -852.8644
## Nagelkerke R2:  0.003245846
## % pres/err predicted correctly:  -405.6968
## % of predictable range [ (model-null)/(1-null) ]:  0.001067609
## ***************************
## model index:  14
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
```

```
##
## Coefficients:
## (Intercept)
##       2.971
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4388 Residual
## Null Deviance:       1710
## Residual Deviance: 1710  AIC: 1712
## log likelihood:  -855.1644
## Nagelkerke R2:  -6.880197e-16
## % pres/err predicted correctly:  -406.1315
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
## model index:  15
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##     3.18715     -0.02811
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4387 Residual
## Null Deviance:       1710
## Residual Deviance: 1710  AIC: 1714
## log likelihood:  -854.9369
## Nagelkerke R2:  0.0003212711
## % pres/err predicted correctly:  -406.0896
## % of predictable range [ (model-null)/(1-null) ]:  0.0001027109
## ***************************
```

```
BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                      AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2

FLPAICSummary <- merge(FLPAICSummary,FLPRes$CoefficientValues,
                       by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                               RunLabel,"_",CurTask,
                               "_freq_len_pos_model_summary.csv"),row.names = FALSE)
kable(FLPAICSummary)
```

Model	AIC	Delta_AIC	AICcWt	nagR2	(Intercept)	stimlen	log_freq	pos	stimlen:log_freq	stimlen:pos	log_freq:pos	I(pos^2)	stimlen:I(pos^2)	log_freq:I(pos^2)	pos:I(pos^2)	stimlen:log_freq:pos	stimlen:log_freq:I(pos^2)	stimlen:pos:I(pos^2)	log_freq:pos:I(pos^2)	stimlen:log_freq:pos:I(pos^2)
preserved ~ stimlen + I(pos^2) + pos + log_freq	1694.1	0.0	0.3	0.1	0.4	0.1	0.1	0.1	NA	NA	NA	0.3	0.4	NA	- 0.0388630	NA	NA	NA	NA	
preserved ~ (I(pos^2) + pos) * log_freq	1694.2	0.1	0.3	0.1	0.4	0.1	0.1	0.1	- 0.0009750	NA	NA	0.4	0.4	NA	- 0.0430607	- 0.0391226	NA	NA	NA	
preserved ~ stimlen * log_freq + I(pos^2) + pos	1695.1	0.9	0.2	0.1	0.4	0.1	0.1	0.1	0.0082739	NA	NA	0.4	0.4	NA	- 0.0391469	NA	NA	NA	NA	
preserved ~ stimlen + (I(pos^2) + pos) * log_freq	1696.2	2.0	0.1	0.1	0.4	0.1	0.1	0.1	NA	NA	NA	0.4	0.4	NA	- 0.0432606	- 0.0391389	NA	NA	NA	
preserved ~ pos + log_freq	1697.3	3.1	0.1	0.1	0.4	0.1	0.1	0.1	0.1484831	NA	NA	0.0	0.0	NA	NA	NA	NA	NA	NA	
preserved ~ stimlen * log_freq + (I(pos^2) + pos) * log_freq	1698.3	4.1	0.1	0.1	0.4	0.1	0.1	0.1	0.0107891	0.1037349	0.0	0.4	0.4	0.0860976	- 0.0430150	NA	- 0.0086793	NA	NA	
preserved ~ pos * log_freq	1698.4	4.2	0.1	0.1	0.4	0.1	0.1	0.1	0.0933396	NA	NA	0.0	0.0	0.0787974	NA	NA	NA	NA	NA	
preserved ~ stimlen + pos + log_freq	1699.5	5.3	0.0	0.1	0.4	0.1	0.1	0.1	0.1450842	NA	NA	0.0	0.0	0.0722062	NA	NA	NA	NA	NA	
preserved ~ stimlen + pos * log_freq	1700.6	6.4	0.0	0.1	0.4	0.1	0.1	0.1	0.0870878	NA	NA	0.0	0.0	0.0839461	NA	NA	NA	NA	NA	
preserved ~ stimlen * log_freq + pos	1701.7	7.5	0.0	0.1	0.4	0.1	0.1	0.1	0.1872091	0.0054495	NA	0.0	0.0	0.0722126	NA	NA	NA	NA	NA	
preserved ~ stimlen + log_freq	1701.7	7.5	0.0	0.1	0.4	0.1	0.1	0.1	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	
preserved ~ stimlen * log_freq + pos * log_freq	1702.2	8.0	0.0	0.1	0.4	0.1	0.1	0.1	0.1832163	0.0143555	NA	0.0	0.0	0.0863891	0.0202411	NA	NA	NA	NA	
preserved ~ stimlen * log_freq	1703.5	9.3	0.0	0.1	0.4	0.1	0.1	0.1	0.1868857	0.0054214	NA	NA	NA	NA	NA	NA	NA	NA	NA	

Model	AIC DeltaAIC AICexp AICw NagR2 (Intercept) [illegible, overprinted]											stimlen:I(pos^2)
preserved ~ I(pos^2) + pos	1705.[illegible]	NA	NA	NA	0.3979344	NA	NA	-0.0404221	NA	NA	NA	NA
preserved ~ stimlen + I(pos^2) + pos	1706.[illegible]	-0.0423187	NA	NA	0.3913865	NA	NA	-0.0381829	NA	NA	NA	NA
preserved ~ stimlen * (I(pos^2) + pos)	1707.[illegible]	[illegible]	NA	NA	1.5354032	NA	NA	-0.1751118	NA	NA	-0.1399350	0.0163056
preserved ~ stimlen * pos	1708.[illegible]	[illegible]	NA	NA	0.5214927	NA	NA	NA	NA	NA	-0.0525769	NA
preserved ~ pos	1711.[illegible]	NA	NA	NA	0.0545009	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen + pos	1711.[illegible]	-0.0616753	NA	NA	0.0722129	NA	NA	NA	NA	NA	NA	NA
preserved ~ 1	1712.[illegible]	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen	1713.[illegible]	-0.0281072	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA

```r
print(BestFLPModelFormula)
```

```
## [1] "preserved ~ stimlen + I(pos^2) + pos + log_freq"
```

```r
print(BestFLPModel)
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos     log_freq
##    2.218823     0.004615    -0.038863     0.396946     0.146144
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4384 Residual
## Null Deviance:       1710
## Residual Deviance: 1684  AIC: 1694
```

```r
# do a median split on frequency to plot hf/ls effects (analysis is continuous)
median_freq <- median(PosDat$log_freq)
PosDat$freq_bin[PosDat$log_freq >= median_freq]<-"hf"
PosDat$freq_bin[PosDat$log_freq <  median_freq]<-"lf"
```

```r
PosDat$FLPFitted<-fitted(BestFLPModel)

HFDat <- PosDat[PosDat$freq_bin == "hf",]
LFDat <- PosDat[PosDat$freq_bin == "lf",]

HF_Plot <- plot_len_pos_obs_predicted(HFDat,paste0(CurPat," - ",RunLabel," - High frequency"),"FLPFitte
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
LF_Plot <- plot_len_pos_obs_predicted(LFDat,paste0(CurPat," - ",RunLabel," - Low frequency"),"FLPFitted
```

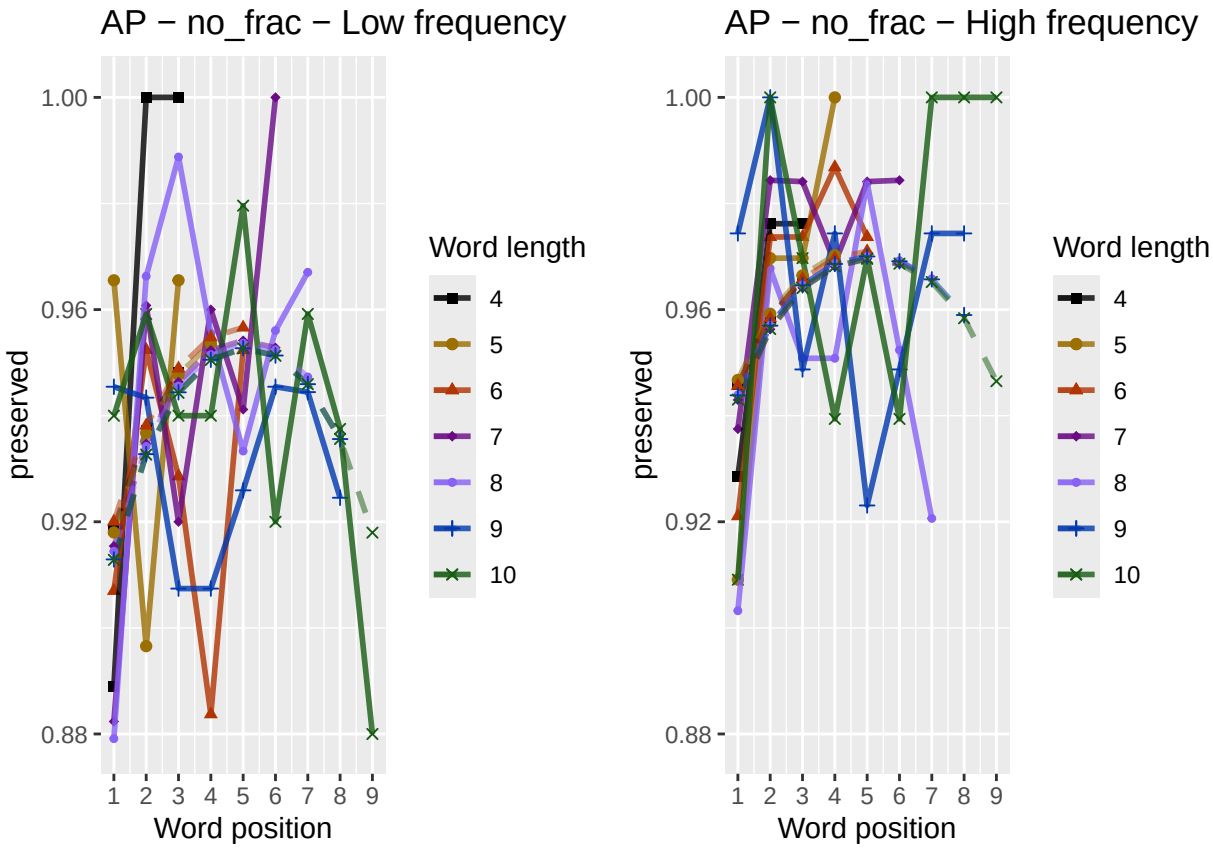
```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
library(ggpubr)
Both_Plots <- ggarrange(LF_Plot,HF_Plot) # labels=c("LF","HF",ncol=2)
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_point()`).
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_line()`).
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",
              CurTask,"_frequency_effect_length_pos_wfit.png"),
       device="png",unit="cm",width=30,height=11)
print(Both_Plots)
```




```
# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",
  "preserved ~ pos",
  "preserved ~ stimlen",
  "preserved ~ 1"
)
```

```
MERes<-TestModels(MEModelEquations,PosDat)
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##      3.2029      -0.9479
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4387 Residual
## Null Deviance:       1710
## Residual Deviance: 1648  AIC: 1652
## log likelihood:  -823.778
## Nagelkerke R2:  0.04400125
## % pres/err predicted correctly:  -396.6488
## % of predictable range [ (model-null)/(1-null) ]:  0.02329142
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.5409       0.1848
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4387 Residual
## Null Deviance:       1710
## Residual Deviance: 1684  AIC: 1688
## log likelihood:  -842.159
## Nagelkerke R2:  0.01830895
## % pres/err predicted correctly:  -403.6493
## % of predictable range [ (model-null)/(1-null) ]:  0.006096759
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     2.24753     -0.04042      0.39793
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4386 Residual
## Null Deviance:       1710
## Residual Deviance: 1699  AIC: 1705
## log likelihood:  -849.7454
## Nagelkerke R2:  0.007642074
## % pres/err predicted correctly:  -405.0016
## % of predictable range [ (model-null)/(1-null) ]:  0.002775195
## ***************************
```

```
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      2.7682       0.0545
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4387 Residual
## Null Deviance:       1710
## Residual Deviance: 1708  AIC: 1712
## log likelihood:  -853.8292
## Nagelkerke R2:  0.0018847
## % pres/err predicted correctly:  -405.8631
## % of predictable range [ (model-null)/(1-null) ]:  0.0006591293
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       2.971
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4388 Residual
## Null Deviance:       1710
## Residual Deviance: 1710  AIC: 1712
## log likelihood:  -855.1644
## Nagelkerke R2:  -6.880197e-16
## % pres/err predicted correctly:  -406.1315
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##     3.18715     -0.02811
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4387 Residual
## Null Deviance:       1710
## Residual Deviance: 1710  AIC: 1714
## log likelihood:  -854.9369
## Nagelkerke R2:  0.0003212711
## % pres/err predicted correctly:  -406.0896
## % of predictable range [ (model-null)/(1-null) ]:  0.0001027109
## ***************************
```

```
BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]
```

```
MEAICSummary<-data.frame(Model=MERes$Model,
                         AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                          by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                              CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1651.556	0.00000	1	1	0.0440013	3.202945	NA	-0.9478552	NA	NA	NA
preserved ~ CumPres	1688.318	36.76201	0	0	0.0183089	2.540926	0.1848195	NA	NA	NA	NA
preserved ~ (I(pos^2) + pos)	1705.491	53.93478	0	0	0.0076422	2.247529	NA	NA	-0.0404221	0.3979344	NA
preserved ~ pos	1711.658	60.10245	0	0	0.0018847	2.768238	NA	NA	NA	0.0545009	NA
preserved ~ 1	1712.329	60.77286	0	0	0.0000000	2.970894	NA	NA	NA	NA	NA
preserved ~ stimlen	1713.874	62.31777	0	0	0.0003213	3.187147	NA	NA	NA	NA	-0.0281072

```
if(DoSimulations){
      BestMEModelFormulaRnd <- BestMEModelFormula
    if(grepl("CumPres",BestMEModelFormulaRnd)){
      BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
    }else if(grepl("CumErr",BestMEModelFormulaRnd)){
      BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
    }

    RndModelAIC<-numeric(length=RandomSamples)
    for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
      PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
      PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
      BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                          family="binomial",data=PosDat)
      RndModelAIC[rindex] <- BestModelRnd$aic
    }
    ModelNames<-c(paste0("***",BestMEModelFormula),
                  rep(BestMEModelFormulaRnd,RandomSamples))
    AICValues <- c(BestMEModel$aic,RndModelAIC)
    BestMEModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
    BestMEModelRndDF <- BestMEModelRndDF %>% arrange(AIC)
    BestMEModelRndDF <- rbind(BestMEModelRndDF,
```

```
                              data.frame(Name=c("Random average"),
                                         AIC=c(mean(RndModelAIC))))
    BestMEModelRndDF <- rbind(BestMEModelRndDF,
                              data.frame(Name=c("Random SD"),
                                         AIC=c(sd(RndModelAIC))))

    write.csv(BestMEModelRndDF,
              paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                     "_best_main_effects_model_with_random_cum_term.csv"),
              row.names = FALSE)
}
```

```
syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary,paste0(TablesDir,CurPat,"_",
                                        RunLabel,"_",CurTask,
                                        "_syllable_component_summary.csv"),row.names = FALSE)
kable(syll_component_summary)
```

syll_component	MeanPres	N
1	0.9721739	575
O	0.9360551	2033
P	1.0000000	36
S	0.8809524	252
V	0.9745479	1493

```
#  main effects models for data without satellite positions

keep_components = c("O","V","1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                              stimlen,stim,pos,
                              preserved,syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data,MEModelEquations)
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##        3.31        -1.12
##
## Degrees of Freedom: 4100 Total (i.e. Null);  4099 Residual
## Null Deviance:       1502
## Residual Deviance: 1432  AIC: 1436
## log likelihood:  -716.0717
```

```
## Nagelkerke R2:  0.05500584
## % pres/err predicted correctly:  -339.7026
## % of predictable range [ (model-null)/(1-null) ]:  0.03068758
## **************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.5686       0.2313
##
## Degrees of Freedom: 4100 Total (i.e. Null);  4099 Residual
## Null Deviance:       1502
## Residual Deviance: 1471  AIC: 1475
## log likelihood:  -735.3793
## Nagelkerke R2:  0.0246756
## % pres/err predicted correctly:  -347.6797
## % of predictable range [ (model-null)/(1-null) ]:  0.007992186
## **************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     2.10692     -0.05535      0.53700
##
## Degrees of Freedom: 4100 Total (i.e. Null);  4098 Residual
## Null Deviance:       1502
## Residual Deviance: 1485  AIC: 1491
## log likelihood:  -742.352
## Nagelkerke R2:  0.01365193
## % pres/err predicted correctly:  -348.7924
## % of predictable range [ (model-null)/(1-null) ]:  0.004826646
## **************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##     2.81047      0.06663
##
## Degrees of Freedom: 4100 Total (i.e. Null);  4099 Residual
## Null Deviance:       1502
## Residual Deviance: 1498  AIC: 1502
## log likelihood:  -749.2363
## Nagelkerke R2:  0.002731093
## % pres/err predicted correctly:  -350.1582
## % of predictable range [ (model-null)/(1-null) ]:  0.0009409054
```

```
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       3.058
##
## Degrees of Freedom: 4100 Total (i.e. Null);  4100 Residual
## Null Deviance:       1502
## Residual Deviance: 1502  AIC: 1504
## log likelihood:  -750.9543
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -350.4889
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##    3.118559    -0.007873
##
## Degrees of Freedom: 4100 Total (i.e. Null);  4099 Residual
## Null Deviance:       1502
## Residual Deviance: 1502  AIC: 1506
## log likelihood:  -750.9388
## Nagelkerke R2:  2.478146e-05
## % pres/err predicted correctly:  -350.4863
## % of predictable range [ (model-null)/(1-null) ]:  7.54809e-06
## ***************************
```

```
write.csv(SimpSyllMEAICSummary,
          paste0(TablesDir,CurPat,"_",RunLabel,"_",
                 CurTask,
                 "_OV1_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1436.143	0.00000	1	1	0.055005	8.309773	NA	-1.120215	NA	NA	NA
preserved ~ CumPres	1474.759	38.61534	0	0	0.024675	6.568635	0.2313235	NA	NA	NA	NA
preserved ~ (I(pos^2) + pos)	1490.704	54.56066	0	0	0.013651	2.106922	NA	NA	-0.0553549	0.5369974	NA
preserved ~ pos	1502.473	66.32928	0	0	0.002731	2.810471	NA	NA	NA	0.0666282	NA
preserved ~ 1	1503.909	67.76534	0	0	0.000000	3.058146	NA	NA	NA	NA	NA
preserved ~ stimlen	1505.878	69.73417	0	0	0.000024	3.118559	NA	NA	NA	NA	-0.007873

```r
#  main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
#  also reduces data)

keep_components = c("O","V")
OVData <- PosDat[PosDat$syll_component %in% keep_components,]
OVData <- OVData %>% select(stim_number,
                            stimlen,stim,pos,
                            preserved,syll_component)
OVData$CumPres <- CalcCumPres(OVData)
OVData$CumErr <- CalcCumErrFromPreserved(OVData)

SimpSyllMEAICSummary2<-EvaluateSubsetData(OVData,MEModelEquations)
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##       3.184       -1.026
##
## Degrees of Freedom: 3525 Total (i.e. Null);  3524 Residual
## Null Deviance:       1351
## Residual Deviance: 1311  AIC: 1315
## log likelihood:  -655.542
## Nagelkerke R2:  0.03505124
## % pres/err predicted correctly:  -313.6036
## % of predictable range [ (model-null)/(1-null) ]:  0.01683586
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.5832       0.2283
##
## Degrees of Freedom: 3525 Total (i.e. Null);  3524 Residual
## Null Deviance:       1351
## Residual Deviance: 1329  AIC: 1333
## log likelihood:  -664.6177
## Nagelkerke R2:  0.01901352
## % pres/err predicted correctly:  -316.9135
## % of predictable range [ (model-null)/(1-null) ]:  0.006492222
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
```

```
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     2.04472     -0.05489      0.54045
##
## Degrees of Freedom: 3525 Total (i.e. Null);  3523 Residual
## Null Deviance:       1351
## Residual Deviance: 1333  AIC: 1339
## log likelihood:  -666.6625
## Nagelkerke R2:  0.01538865
## % pres/err predicted correctly:  -317.2324
## % of predictable range [ (model-null)/(1-null) ]:  0.005495446
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      2.7079       0.0784
##
## Degrees of Freedom: 3525 Total (i.e. Null);  3524 Residual
## Null Deviance:       1351
## Residual Deviance: 1346  AIC: 1350
## log likelihood:  -673.0328
## Nagelkerke R2:  0.004069057
## % pres/err predicted correctly:  -318.5288
## % of predictable range [ (model-null)/(1-null) ]:  0.001444271
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       2.995
##
## Degrees of Freedom: 3525 Total (i.e. Null);  3525 Residual
## Null Deviance:       1351
## Residual Deviance: 1351  AIC: 1353
## log likelihood:  -675.3171
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -318.9909
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##    3.019224    -0.003149
```

```
##
## Degrees of Freedom: 3525 Total (i.e. Null);  3524 Residual
## Null Deviance:       1351
## Residual Deviance: 1351  AIC: 1355
## log likelihood:  -675.3148
## Nagelkerke R2:  4.102019e-06
## % pres/err predicted correctly:  -318.9905
## % of predictable range [ (model-null)/(1-null) ]:  1.280124e-06
## ***************************
```

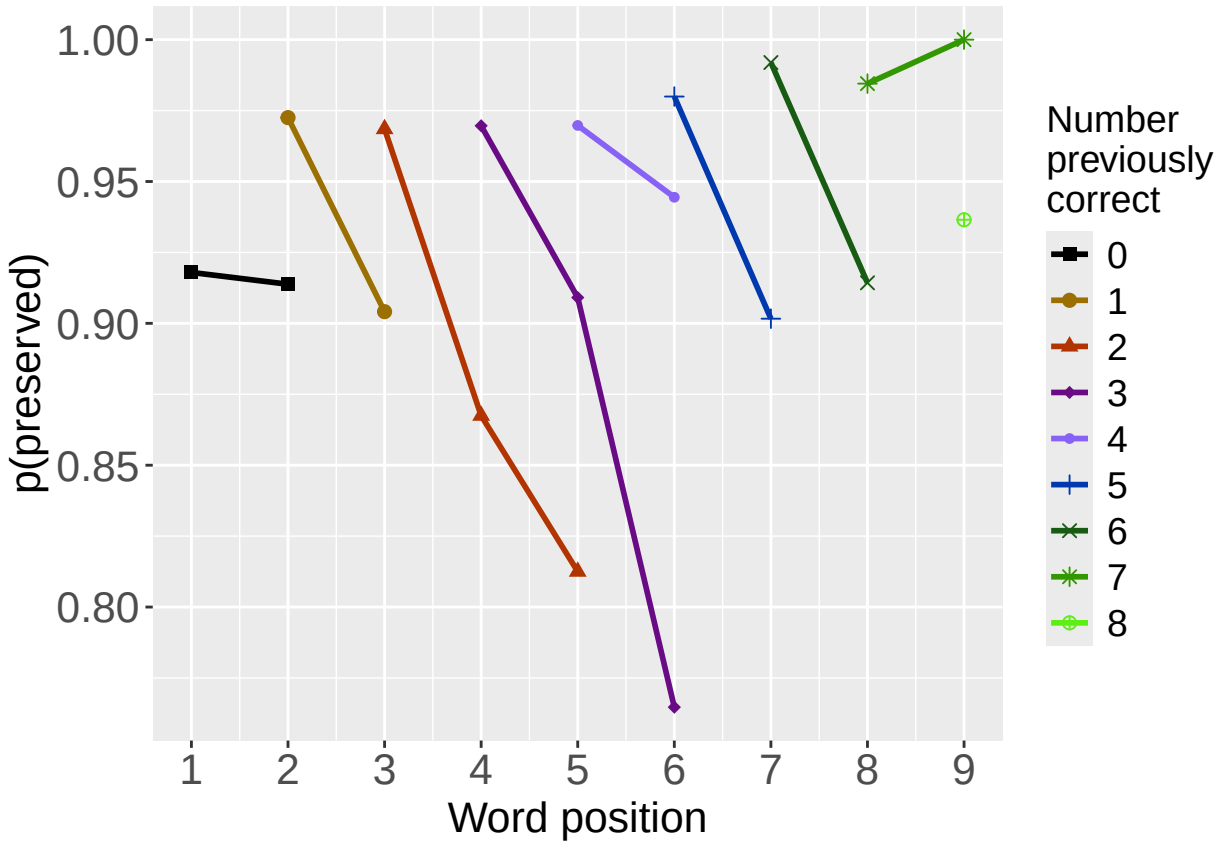
```
write.csv(SimpSyllMEAICSummary2,
          paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                 "_OV_main_effects_model_summary.csv"),row.names = FALSE)
kable(SimpSyllMEAICSummary2)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1315.084	0.00000	1.000000	0.9998800	0.0350513	2.184113	NA	-1.025773	NA	NA	NA
preserved ~ CumPres	1333.235	18.15127	0.000114	0.0001140	0.0190132	2.583187	0.2283337	NA	NA	NA	NA
preserved ~ (I(pos^2) + pos)	1339.325	24.24093	0.000005	0.0000054	0.0153887	2.044717	NA	NA	-0.054892	0.5404461	NA
preserved ~ pos	1350.066	34.98152	0.000000	0.0000000	0.0040692	1.707861	NA	NA	NA	0.0784031	NA
preserved ~ 1	1352.634	37.55015	0.000000	0.0000000	0.0000000	2.995137	NA	NA	NA	NA	NA
preserved ~ stimlen	1354.630	39.54555	0.000000	0.0000000	0.0000043	3.019224	NA	NA	NA	NA	-0.0031485

```
# plot prev err and prev cor plots
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

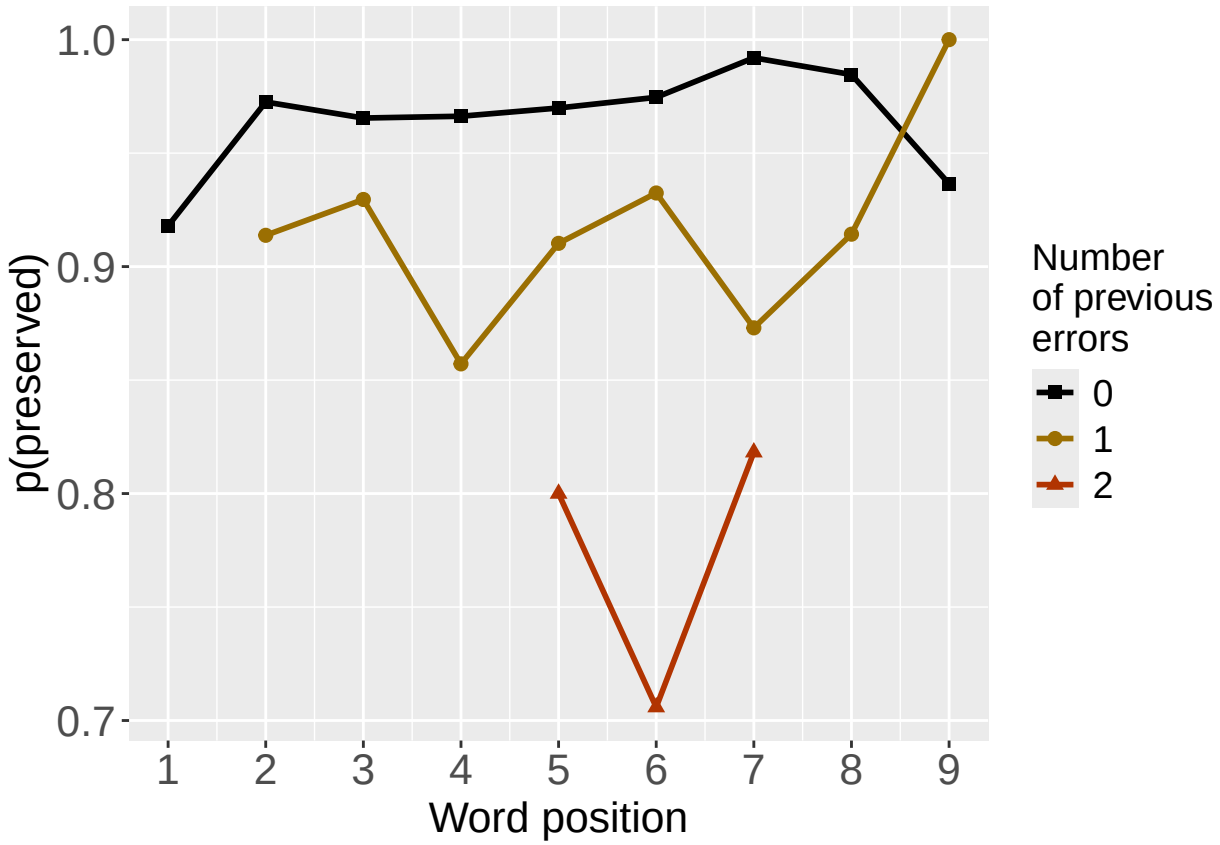
```
print(PrevCorPlot)
```




```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```




```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

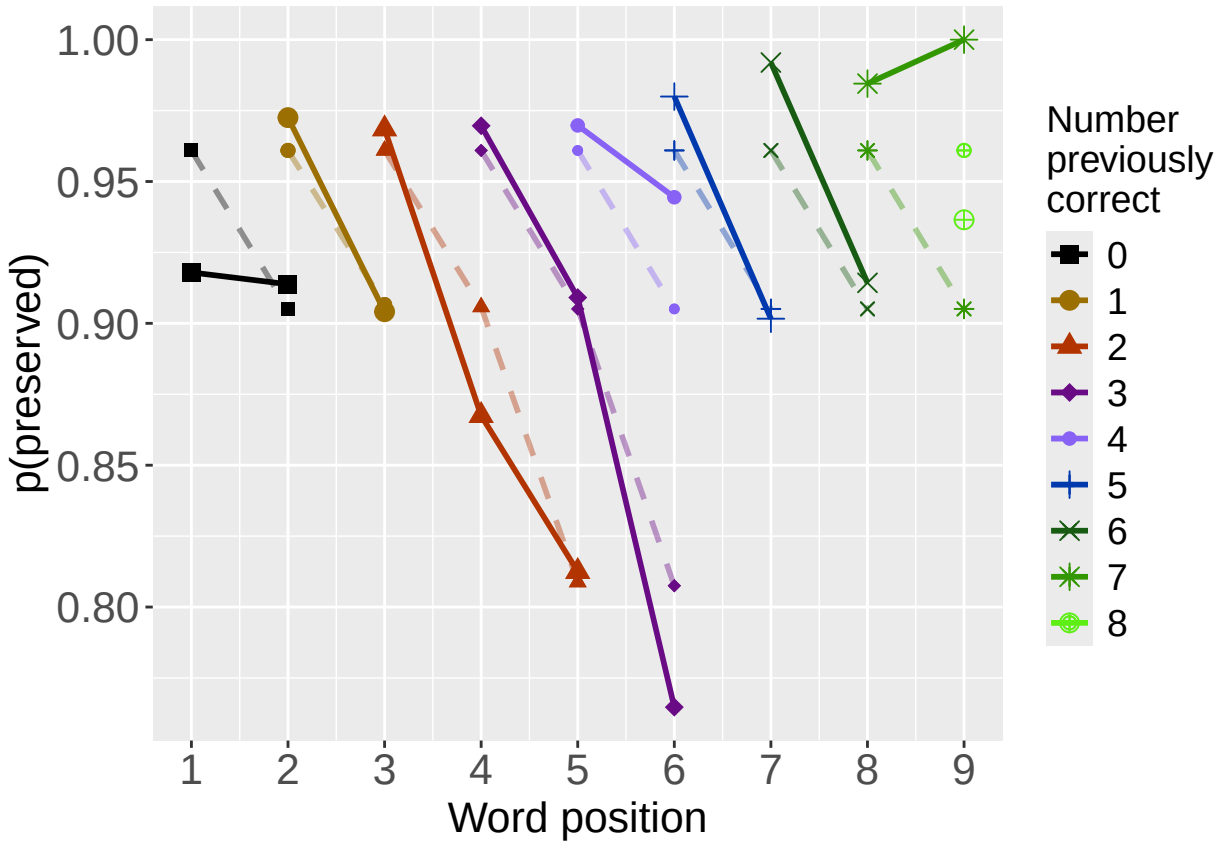
```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

```
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

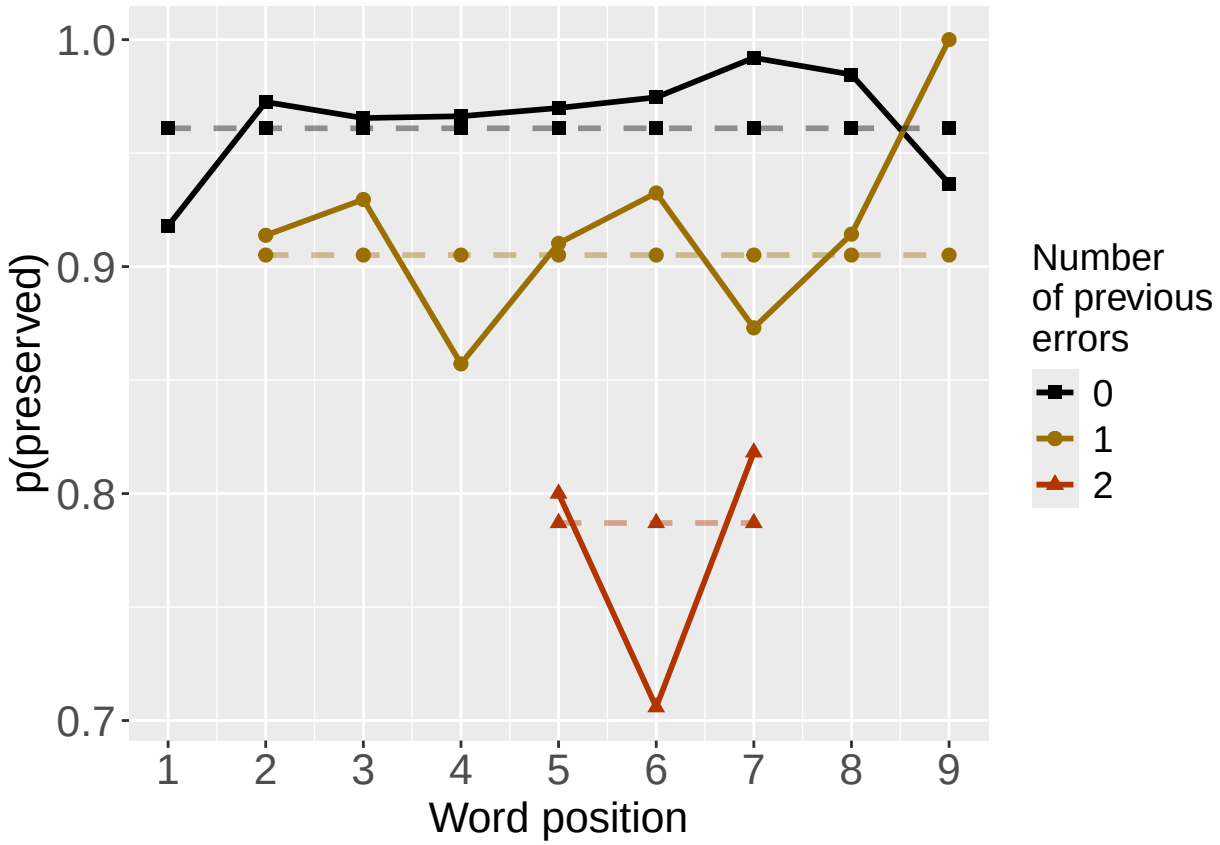
```
print(PrevCorPlot)
```




```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```




```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                 CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                 CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

```r
CumAICSummary <- NULL
```

```r
########
# level 2 -- Add position squared (quadratic with position)
########

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr     I(pos^2)          pos
##     2.14576     -1.14575     -0.04241      0.51244
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4385 Residual
## Null Deviance:       1710
## Residual Deviance: 1621  AIC: 1629
## log likelihood:  -810.7451
## Nagelkerke R2:  0.06208822
## % pres/err predicted correctly:  -394.2644
## % of predictable range [ (model-null)/(1-null) ]:  0.02914808
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##      3.2029      -0.9479
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4387 Residual
## Null Deviance:       1710
## Residual Deviance: 1648  AIC: 1652
## log likelihood:  -823.778
## Nagelkerke R2:  0.04400125
## % pres/err predicted correctly:  -396.6488
## % of predictable range [ (model-null)/(1-null) ]:  0.02329142
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     2.24753     -0.04042      0.39793
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4386 Residual
## Null Deviance:       1710
## Residual Deviance: 1699  AIC: 1705
## log likelihood:  -849.7454
## Nagelkerke R2:  0.007642074
## % pres/err predicted correctly:  -405.0016
## % of predictable range [ (model-null)/(1-null) ]:  0.002775195
## ***************************
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos
preserved ~ CumErr + I(pos^2) + pos	1629.490	0.00000	1.00e+00	0.9999838	0.0620882	2.145758	-1.1457540	-0.0424102	0.5124381
preserved ~ CumErr	1651.556	22.06582	1.62e-05	0.0000162	0.0440012	3.202945	-0.9478552	NA	NA
preserved ~ I(pos^2) + pos	1705.491	76.00060	0.00e+00	0.0000000	0.0076421	2.247529	NA	-0.0404221	0.3979344

```
########
# level 2 -- Add length
########

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##      3.2029      -0.9479
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4387 Residual
## Null Deviance:       1710
## Residual Deviance: 1648  AIC: 1652
## log likelihood:  -823.778
## Nagelkerke R2:  0.04400125
## % pres/err predicted correctly:  -396.6488
## % of predictable range [ (model-null)/(1-null) ]:  0.02329142
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr      stimlen
##    3.165727    -0.949410     0.004892
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4386 Residual
## Null Deviance:       1710
## Residual Deviance: 1648  AIC: 1654
## log likelihood:  -823.7714
## Nagelkerke R2:  0.04401036
## % pres/err predicted correctly:  -396.6383
## % of predictable range [ (model-null)/(1-null) ]:  0.02331717
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen
##     3.18715     -0.02811
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4387 Residual
## Null Deviance:       1710
## Residual Deviance: 1710  AIC: 1714
## log likelihood:  -854.9369
## Nagelkerke R2:  0.0003212711
## % pres/err predicted correctly:  -406.0896
## % of predictable range [ (model-null)/(1-null) ]:  0.0001027109
## ***************************
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	stimlen
preserved ~ CumErr	1651.556	0.000000	1.0000000	0.72977	0.0440012	3.202945	-0.9478552	NA
preserved ~ CumErr + stimlen	1653.543	1.986911	0.3702948	0.27023	0.0440104	3.165727	-0.9494104	0.0048921
preserved ~ stimlen	1713.874	62.317772	0.0000000	0.00000	0.0003213	3.187147	NA	-0.0281072

```
########
# level 2 -- add cumulative preserved
########

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr      CumPres
##      2.7356      -0.9864       0.2058
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4386 Residual
## Null Deviance:       1710
## Residual Deviance: 1618  AIC: 1624
## log likelihood:  -808.8233
## Nagelkerke R2:  0.06474611
## % pres/err predicted correctly:  -393.5642
## % of predictable range [ (model-null)/(1-null) ]:  0.03086792
## ***************************
## model index:  1
##
```

```
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##      3.2029      -0.9479
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4387 Residual
## Null Deviance:       1710
## Residual Deviance: 1648  AIC: 1652
## log likelihood:  -823.778
## Nagelkerke R2:  0.04400125
## % pres/err predicted correctly:  -396.6488
## % of predictable range [ (model-null)/(1-null) ]:  0.02329142
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.5409       0.1848
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4387 Residual
## Null Deviance:       1710
## Residual Deviance: 1684  AIC: 1688
## log likelihood:  -842.159
## Nagelkerke R2:  0.01830895
## % pres/err predicted correctly:  -403.6493
## % of predictable range [ (model-null)/(1-null) ]:  0.006096759
## ***************************
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres
preserved ~ CumErr + CumPres	1623.647	0.00000	1e+00	0.9999991	0.0647461	2.735640	-0.9864123	0.2057928
preserved ~ CumErr	1651.556	27.90927	9e-07	0.0000009	0.0440012	3.202945	-0.9478552	NA
preserved ~ CumPres	1688.318	64.67128	0e+00	0.0000000	0.0183089	2.540926	NA	0.1848195

```
########
# level 2 -- Add linear position (NOT quadratic)
########

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## **************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr          pos
##      2.6818      -1.1428       0.1551
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4386 Residual
## Null Deviance:       1710
## Residual Deviance: 1630  AIC: 1636
## log likelihood:  -814.9301
## Nagelkerke R2:  0.05629195
## % pres/err predicted correctly:  -395.3074
## % of predictable range [ (model-null)/(1-null) ]:  0.02658606
## **************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##      3.2029      -0.9479
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4387 Residual
## Null Deviance:       1710
## Residual Deviance: 1648  AIC: 1652
## log likelihood:  -823.778
## Nagelkerke R2:  0.04400125
## % pres/err predicted correctly:  -396.6488
## % of predictable range [ (model-null)/(1-null) ]:  0.02329142
## **************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      2.7682       0.0545
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4387 Residual
## Null Deviance:       1710
## Residual Deviance: 1708  AIC: 1712
## log likelihood:  -853.8292
## Nagelkerke R2:  0.0018847
## % pres/err predicted correctly:  -405.8631
## % of predictable range [ (model-null)/(1-null) ]:  0.0006591293
## **************************
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	pos
preserved ~ CumErr + pos	1635.860	0.00000	1.0000000	0.9996096	0.0562920	2.681841	-1.1428099	0.1550752
preserved ~ CumErr	1651.556	15.69573	0.0003906	0.0003904	0.0440012	3.202945	-0.9478552	NA
preserved ~ pos	1711.658	75.79817	0.0000000	0.0000000	0.0018847	2.768238	NA	0.0545009

```
CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary,paste0(TablesDir,CurPat,"_",
                               RunLabel,"_",CurTask,
                               "_main_effects_plus_one_model_summary.csv"),row.names = FALSE)
kable(CumAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	stimlen	CumPres
preserved ~ CumErr + CumPres	1623.647	0.000000	1.0000000	0.9999991	0.1064746	2.735640	-0.9864123	NA	NA	NA	0.2057928
preserved ~ CumErr + I(pos^2) + pos	1629.490	0.000000	1.0000000	0.9999838	0.0620882	2.145758	-1.1457540	-0.0424102	0.5124381	NA	NA
preserved ~ CumErr + pos	1635.860	0.000000	1.0000000	0.9996096	0.0562920	2.681841	-1.1428099	NA	0.1550752	NA	NA
preserved ~ CumErr	1651.556	15.695726	0.0003906	0.0003904	0.0440012	3.202945	-0.9478552	NA	NA	NA	NA
preserved ~ CumErr	1651.556	0.000000	1.0000000	0.7297700	0.0440012	3.202945	-0.9478552	NA	NA	NA	NA
preserved ~ CumErr	1651.556	27.909270	1.0000000	0.0000000	0.0440012	3.202945	-0.9478552	NA	NA	NA	NA
preserved ~ CumErr	1651.556	15.695726	0.0003906	0.0003904	0.0440012	3.202945	-0.9478552	NA	NA	NA	NA
preserved ~ CumErr + stimlen	1653.543	3.986911	0.3702948	0.2702300	0.0440104	3.165727	-0.9494104	NA	NA	0.0048921	NA
preserved ~ CumPres	1688.318	64.671283	0.0000000	0.0000000	0.0183089	2.540926	NA	NA	NA	NA	0.1848195
preserved ~ I(pos^2) + pos	1705.497	76.000602	0.0000000	0.0000000	0.0076422	1.247529	NA	-0.0404221	0.3979344	NA	NA
preserved ~ pos	1711.658	75.798170	1.0000000	0.0000000	0.0018847	2.768238	NA	NA	0.0545009	NA	NA
preserved ~ stimlen	1713.876	62.317772	0.0000000	0.0000000	0.0003213	3.187147	NA	NA	NA	-0.0281072	NA

```
# explore influence of frequency and length

if(grepl("stimlen",BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
      BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
      paste0(BestModelFormulaL2," + log_freq")
  )
}else if(grepl("log_freq",BestModelFormulaL2)){ # if log_freq is already in the formula
    Level3ModelEquations<-c(
```

```
      BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
      paste0(BestModelFormulaL2," + stimlen")
  )
}else{
  Level3ModelEquations<-c(
      BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
      paste0(BestModelFormulaL2," + log_freq"),
      paste0(BestModelFormulaL2," + stimlen"),
      paste0(BestModelFormulaL2," + stimlen + log_freq")
  )
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)
```

```
## ****************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr      CumPres     log_freq
##      2.7278      -0.9424       0.2169       0.1304
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4385 Residual
## Null Deviance:       1710
## Residual Deviance: 1606  AIC: 1614
## log likelihood:  -802.8208
## Nagelkerke R2:  0.07303311
## % pres/err predicted correctly:  -392.9295
## % of predictable range [ (model-null)/(1-null) ]:  0.03242689
## ****************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr      CumPres      stimlen     log_freq
##     3.06584     -0.93576      0.22864     -0.04816      0.11903
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4384 Residual
## Null Deviance:       1710
## Residual Deviance: 1605  AIC: 1615
## log likelihood:  -802.288
## Nagelkerke R2:  0.07376759
## % pres/err predicted correctly:  -392.8886
## % of predictable range [ (model-null)/(1-null) ]:  0.03252726
## ****************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr      CumPres      stimlen
##     3.34506     -0.96751      0.22972     -0.08721
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4385 Residual
## Null Deviance:       1710
## Residual Deviance: 1614  AIC: 1622
## log likelihood:  -806.9026
## Nagelkerke R2:  0.06740026
## % pres/err predicted correctly:  -393.3999
## % of predictable range [ (model-null)/(1-null) ]:  0.03127139
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr      CumPres
##      2.7356      -0.9864       0.2058
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4386 Residual
## Null Deviance:       1710
## Residual Deviance: 1618  AIC: 1624
## log likelihood:  -808.8233
## Nagelkerke R2:  0.06474611
## % pres/err predicted correctly:  -393.5642
## % of predictable range [ (model-null)/(1-null) ]:  0.03086792
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##       2.971
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4388 Residual
## Null Deviance:       1710
## Residual Deviance: 1710  AIC: 1712
## log likelihood:  -855.1644
## Nagelkerke R2:  -6.880197e-16
## % pres/err predicted correctly:  -406.1315
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
```

```
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

AICSummary<-data.frame(Model=Level3Res$Model,
                       AIC=Level3Res$AIC,
```

```
                        row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                          by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                            CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres	log_freq	stimlen
preserved ~ CumErr + CumPres + log_freq	1613.642	0.0000000	1.0000000	0.6059319	0.9073033	2.727751	-0.9424027	0.2168508	0.1303755	NA
preserved ~ CumErr + CumPres + stimlen + log_freq	1614.576	0.9344025	0.6267539	0.3797770	0.9073767	3.065844	-0.9357627	0.2286381	0.1190287	-0.0481572
preserved ~ CumErr + CumPres + stimlen	1621.805	8.1637250	0.0168760	0.0102257	0.9067400	3.345061	-0.9675136	0.2297196	NA	-0.0872079
preserved ~ CumErr + CumPres	1623.647	10.0051396	0.0067207	0.0040728	0.9064746	2.735640	-0.9864123	0.2057928	NA	NA
preserved ~ 1	1712.329	98.6872772	0.0000000	0.0000000	0.0000000	0.970894	NA	NA	NA	NA

```
# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[BestModelIndexL3]

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions
```

```
## Single term deletions
##
## Model:
## preserved ~ CumErr + CumPres + log_freq
##          Df Deviance    AIC
## CumErr    1   1666.2 1672.2
## CumPres   1   1638.8 1644.8
## log_freq  1   1617.7 1623.7
## <none>        1605.6 1613.6
```

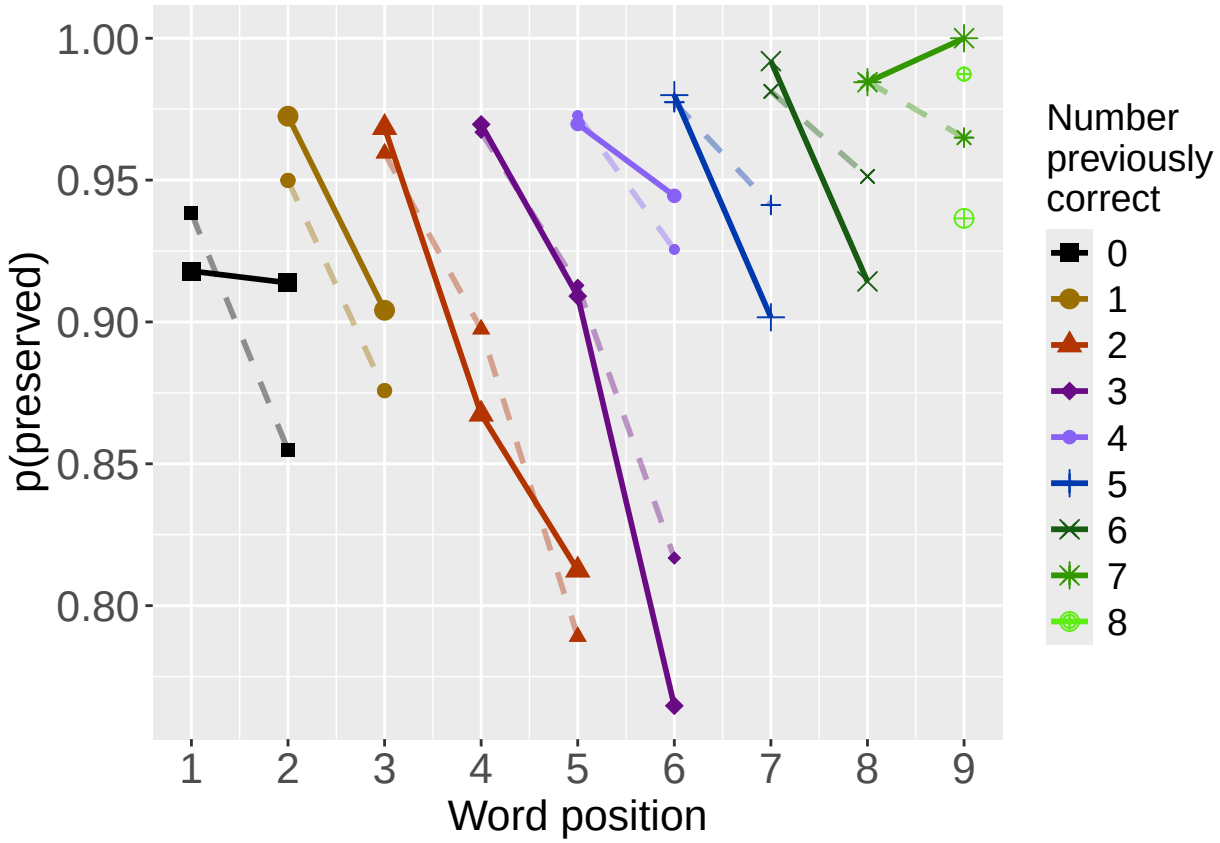
```
################################
# Single deletions from best model
################################
```

```
write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                    RunLabel,"_",CurTask,
                                    "_best_model_single_term_deletions.csv"),
          row.names = TRUE)
```

```
# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

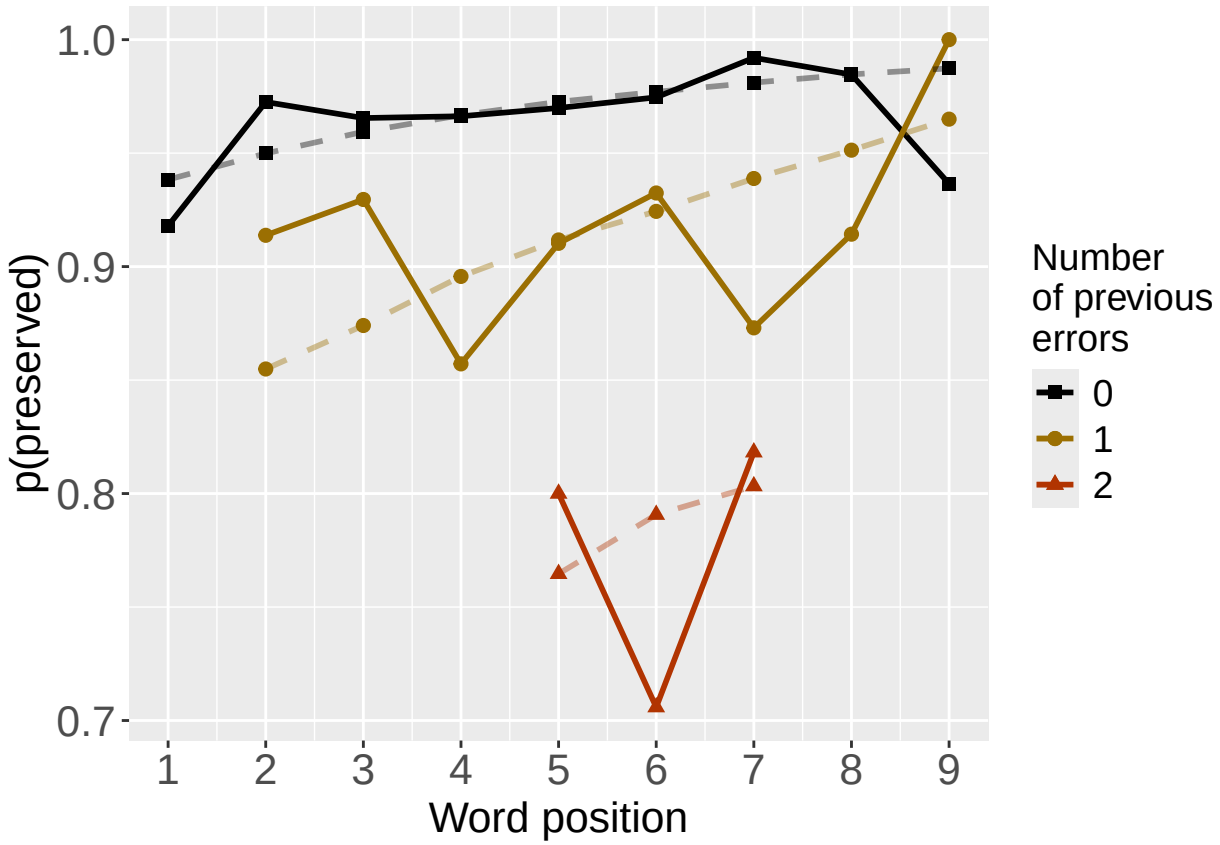
```
print(PrevCorPlot)
```




```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```




```
PlotName<-paste0(FigDir,CurPat,"_",
                 RunLabel,"_",CurTask,
                 "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression =
```

```
## Saving 6.5 x 4.5 in image
```

```
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "
```

```
## Saving 6.5 x 4.5 in image
```

```
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
  # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
                        family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
```

```
  }
  ModelNames<-c(paste0("***",BestModelFormulaL3),
                rep(BestModelFormulaL3Rnd,RandomSamples))
  AICValues <- c(BestModelL3$aic,RndModelAIC)
  BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
  BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
  BestModelRndDF <- rbind(BestModelRndDF,
                          data.frame(Name=c("Random average"),
                                     AIC=c(mean(RndModelAIC))))
  BestModelRndDF <- rbind(BestModelRndDF,
                          data.frame(Name=c("Random SD"),
                                     AIC=c(sd(RndModelAIC))))
  write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                  "_best_model_with_random_cum_term.csv"),
            row.names = FALSE)
}
```

```
# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)
```

```
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##      3.2029      -0.9479
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4387 Residual
## Null Deviance:       1710
## Residual Deviance: 1648  AIC: 1652
## log likelihood:  -823.778
## Nagelkerke R2:  0.04400125
## % pres/err predicted correctly:  -396.6488
## % of predictable range [ (model-null)/(1-null) ]:  0.02329142
```

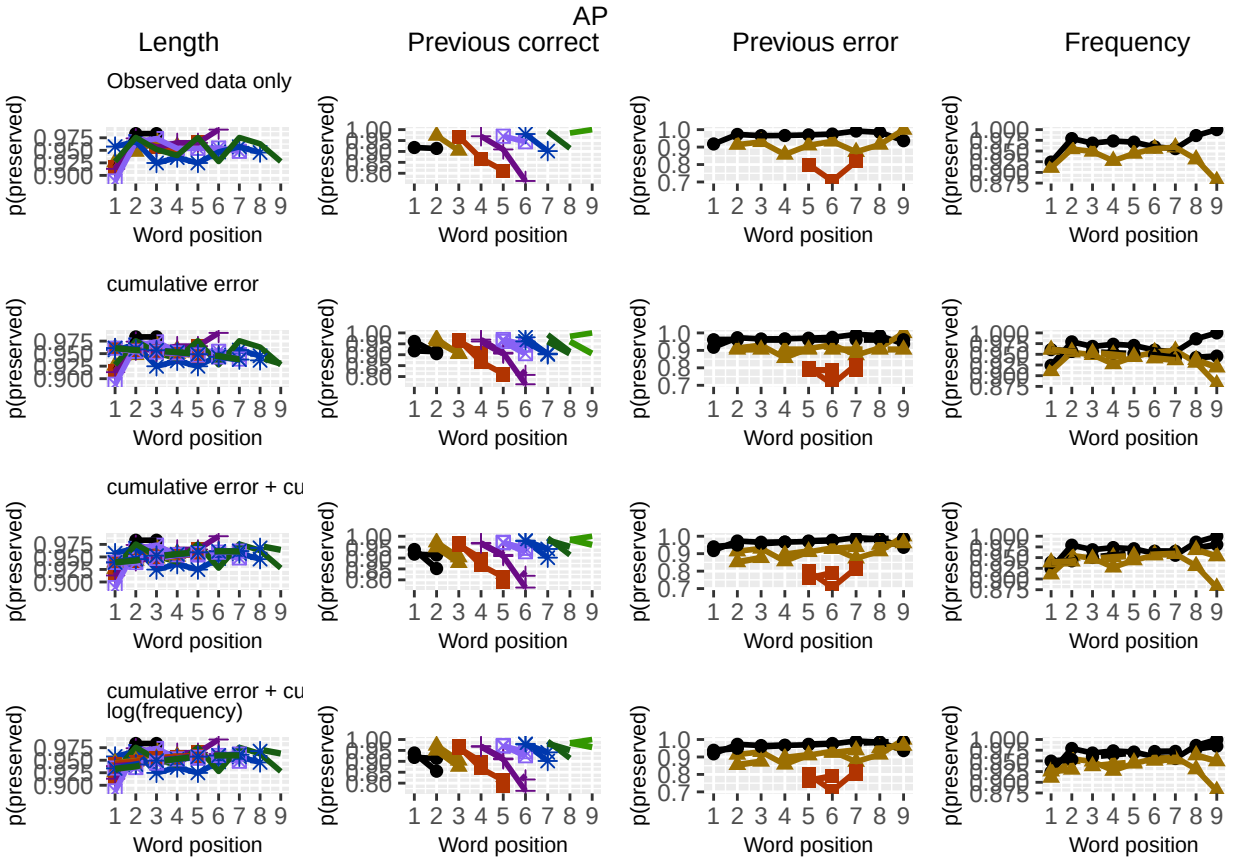
```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr      CumPres
##      2.7356      -0.9864       0.2058
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4386 Residual
## Null Deviance:       1710
## Residual Deviance: 1618  AIC: 1624
## log likelihood:  -808.8233
## Nagelkerke R2:  0.06474611
## % pres/err predicted correctly:  -393.5642
## % of predictable range [ (model-null)/(1-null) ]:  0.03086792
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr      CumPres     log_freq
##      2.7278      -0.9424       0.2169       0.1304
##
## Degrees of Freedom: 4388 Total (i.e. Null);  4385 Residual
## Null Deviance:       1710
## Residual Deviance: 1606  AIC: 1614
## log likelihood:  -802.8208
## Nagelkerke R2:  0.07303311
## % pres/err predicted correctly:  -392.9295
## % of predictable range [ (model-null)/(1-null) ]:  0.03242689
## ***************************
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
```

```
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes d
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
```

```
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes d
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`)

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes d
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes d
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes d
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes d
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes d
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes d
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 5 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 5 rows containing missing values or values outside the scale range (`geom_point()`).
```

```r
ggsave(paste0(PlotName,".tif"),plot=FactorPlot,width = 360,height=400,units="mm",device="tiff",compressi
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot
```




```
DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                       CurTask,"_dominance_analysis_table.csv"),
          row.names = TRUE)
kable(DAContributionAverage)
```

	CumErr	CumPres	log_freq
McFadden	0.0360882	0.0173176	0.0078031
SquaredCorrelation	0.0138967	0.0066651	0.0030081
Nagelkerke	0.0138967	0.0066651	0.0030081
Estrella	0.0143337	0.0068849	0.0030943

	deviance	deviance_explained
CumErr + CumPres + log_freq	1605.642	104.68728
CumErr + CumPres	1617.647	92.68214
CumErr	1647.556	62.77286
null	1710.329	0.00000

	percent_explained	percent_of_explained_deviance	increment_in_explained
CumErr + CumPres + log_freq	6.120886	100.00000	11.46762
CumErr + CumPres	5.418966	88.53238	28.57011
CumErr	3.670222	59.96227	59.96227
null	0.000000	NA	0.00000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##                                                 model deviance deviance_explained percent_explained percent_of_explained_deviance
## CumErr + CumPres + log_freq CumErr + CumPres + log_freq 1605.642          104.68728          6.120886                     100.00000
## CumErr + CumPres                      CumErr + CumPres 1617.647           92.68214          5.418966                      88.53238
## CumErr                                          CumErr 1647.556           62.77286          3.670222                      59.96227
## null                                              null 1710.329            0.00000          0.000000                            NA
##                              increment_in_explained
## CumErr + CumPres + log_freq               11.46762
## CumErr + CumPres                          28.57011
## CumErr                                    59.96227
## null                                       0.00000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
```

```r
NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents
```

```
##          Nagelkerke
## CumErr    0.5895947
## CumPres   0.2827813
## log_freq  0.1276240
```

```r
sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length

## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
```

model	p_accounted_for	model_deviance
preserved ~ CumErr	0.7919955	1647.556
preserved ~ CumErr+CumPres+log_freq	0.8421714	1605.642
preserved ~ CumErr+CumPres	0.8431752	1617.647

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length

## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
sse_table<-sse_results_table(sse_results_list)
write.csv(sse_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                           CurTask,"_sse_results_table.csv"),row.names = TRUE)
sse_table
```

```
##                                 model p_accounted_for model_deviance diff_CumErr diff_CumErr+CumPres+log_freq diff_CumErr+CumPres
## 1                   preserved ~ CumErr       0.7919955       1647.556  0.00000000                 -0.050175921        -0.051179783
## 2 preserved ~ CumErr+CumPres+log_freq       0.8421714       1605.642  0.05017592                  0.000000000        -0.001003862
## 3          preserved ~ CumErr+CumPres       0.8431752       1617.647  0.05117978                  0.001003862         0.000000000
```

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                   CurTask,"_results_report_df.csv"),row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

model	diff_CumErr	diff_CumErr+CumPres+log_freq	diff_CumErr+CumPres
preserved ~ CumErr	0.0000000	-0.0501759	-0.0511798
preserved ~ CumErr+CumPres+log_freq	0.0501759	0.0000000	-0.0010039
preserved ~ CumErr+CumPres	0.0511798	0.0010039	0.0000000